

DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Building and Evaluating a Multi-Source Query Interface with Unified Operators

Kai Arenja





DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Building and Evaluating a Multi-Source Query Interface with Unified Operators

Aufbau und Bewertung eines Multi-Source-Abfragesystems mit einheitlichen Operatoren

Author:	Kai Arenja
Supervisor:	Wenz, Fabian
Advisor:	Prof. Giceva Makreshanska, Jana
Submission Date:	01.04.2026



I confirm that this bachelor's thesis in informatics is my own work and I have documented all sources and material used.

Munich, 01.04.2026 Kai Arenja

Preface

In the context of this thesis, generative AI tools (e.g., ChatGPT) were used for linguistic refinement and general editing support.

The author retains full responsibility for the content. All generated suggestions were critically reviewed, revised, and scientifically contextualized.

Abstract

Modern organizations increasingly rely on heterogeneous data sources such as relational databases, document repositories, email systems, and web-based resources. Retrieving and combining information across these sources remains challenging due to fragmented workflows, heterogeneous data models, and a trade-off between transparency and usability. Traditional query systems provide precise control but are largely limited to structured data, whereas recent approaches based on large language models and retrieval-augmented generation often lack transparency and user control.

This thesis presents and evaluates **Rubicon**, a unified operator-based query system for transparent and interactive multi-source information retrieval. Rubicon introduces a small set of composable operators, including `FIND`, `FROM`, `WHERE`, `RETURN`, and `?`, enabling users to formulate structured queries across heterogeneous data sources in a consistent and extensible manner. The system is implemented as a modular architecture with source-specific wrappers for relational databases, Wikipedia, email systems, and text corpora. An execution engine processes operator-based queries, integrates intermediate results, and provides an explicit execution trace that exposes all processing steps, supporting transparency and iterative query refinement. The system combines deterministic query execution with controlled use of large language models (LLMs) and retrieval-augmented generation (RAG) capabilities.

The effectiveness of Rubicon is evaluated through a controlled user study comparing the operator-based interface with a conventional manual workflow using separate tools. The evaluation considers task completion time, retrieval performance (measured by precision, recall, and F1 score), usability, cognitive workload, and the influence of user expertise on performance. The results show that the proposed approach improves retrieval performance, usability, and robustness while reducing reliance on prior technical expertise.

Overall, this work demonstrates that structured, operator-based query models provide an effective and transparent alternative to fully automated LLM-based approaches for multi-source information retrieval.

Kurzfassung

Moderne Organisationen nutzen zunehmend heterogene Datenquellen wie relationale Datenbanken, Dokumentenrepositorien, E-Mail-Systeme und webbasierte Ressourcen. Die Abfrage und Kombination von Informationen über diese Quellen hinweg bleibt jedoch eine Herausforderung, die durch fragmentierte Arbeitsabläufe, unterschiedliche Datenmodelle sowie einen Zielkonflikt zwischen Transparenz und Benutzerfreundlichkeit geprägt ist. Traditionelle Abfragesysteme ermöglichen eine präzise Kontrolle, sind jedoch weitgehend auf strukturierte Daten beschränkt, während neuere Ansätze auf Basis von Large Language Models und Retrieval-Augmented Generation häufig eine eingeschränkte Transparenz und geringe Benutzerkontrolle aufweisen.

Diese Arbeit präsentiert und evaluiert **Rubicon**, ein vereinheitlichtes, operatorbasiertes Abfragesystem für transparente und interaktive Informationsgewinnung über mehrere Datenquellen hinweg. Rubicon führt eine kleine Menge kombinierbarer Operatoren ein, darunter `FIND`, `FROM`, `WHERE`, `RETURN` und `?`, die es Nutzern ermöglichen, strukturierte Abfragen über heterogene Datenquellen hinweg konsistent zu formulieren. Das System ist als modulare Architektur mit quellenspezifischen Wrappern für relationale Datenbanken, Wikipedia, E-Mail-Systeme und Textkorpora implementiert. Eine Ausführungsengine verarbeitet operatorbasierte Abfragen, integriert Zwischenergebnisse und stellt eine explizite Ausführungsprotokoll bereit, die alle Verarbeitungsschritte offenlegt und damit Transparenz sowie die iterative Verfeinerung von Abfragen unterstützt. Das System kombiniert deterministische Abfrageausführung mit kontrolliertem Einsatz von LLMs sowie Retrieval-Augmented Generation (RAG).

Die Wirksamkeit von Rubicon wird durch eine kontrollierte Nutzerstudie evaluiert, in der die operatorbasierte Schnittstelle mit einem konventionellen manuellen Arbeitsablauf unter Verwendung separater Werkzeuge verglichen wird. Die Evaluation berücksichtigt Bearbeitungszeit, Retrieval-Performance (gemessen anhand von Precision, Recall und F1-Score), Benutzerfreundlichkeit, kognitive Belastung sowie den Einfluss von Nutzerexpertise. Die Ergebnisse zeigen, dass der vorgeschlagene Ansatz Effizienz, Performance und Benutzerfreundlichkeit verbessert und zugleich Unterschiede zwischen Nutzern mit unterschiedlichem Erfahrungsniveau reduziert.

Insgesamt zeigt diese Arbeit die Machbarkeit eines vereinheitlichten operatorbasierten Abfragemodells, wie es in Rubicon umgesetzt ist, um Ausdrucksstärke, Benutzerfreundlichkeit und Transparenz in der Multi-Source-Informationsgewinnung in Einklang zu bringen.

Contents

Preface **v**

Abstract **vii**

Kurzfassung **ix**

1. Introduction **1**
1.1. Motivation . . . 1
1.2. Problem Statement . . . 2
1.3. Research Objectives and Questions . . . 2
1.4. Contributions . . . 3
1.5. Thesis Structure . . . 4

2. Background **5**
2.1. Enterprise Information Retrieval . . . 5
2.2. Federated Data Systems . . . 6
2.3. Retrieval-Augmented Generation . . . 7
2.4. Agentic Retrieval Systems . . . 8
2.5. Large Language Models in Information Retrieval . . . 9
2.6. Human-in-the-Loop Information Retrieval . . . 10
2.7. Related Systems and Frameworks . . . 11

3. System Design **13**
3.1. System Overview . . . 13
3.1.1. Design Goals . . . 13
3.1.2. System Components . . . 14
3.1.3. End-to-End Query Flow . . . 17
3.2. Agentic Query Language (AQL) . . . 17
3.2.1. Language Design and Concepts . . . 17
3.2.2. Query Structure . . . 18
3.2.3. Example Queries . . . 19
3.3. Query Processing . . . 20
3.3.1. Query Representation and Parsing . . . 20
3.3.2. Execution Model . . . 21
3.3.3. Result Construction . . . 22
3.4. Retrieval Architecture . . . 22
3.4.1. Source Wrappers . . . 22

3.4.2. Wrapper Implementations . . . 23
3.5. Transparency and Execution Trace . . . 28
3.5.1. Execution Trace Model . . . 28
3.5.2. Comparison to Black-Box Retrieval Systems . . . 28
3.6. Design Rationale . . . 29
3.6.1. Limitations of Fully Agentic Retrieval . . . 29
3.6.2. Model Selection and Retrieval Strategy . . . 30
3.6.3. Why Operator-Based Control . . . 31
3.6.4. Why Transparent Execution . . . 31

4. Evaluation **33**
4.1. Evaluation Goals . . . 33
4.2. User Study Design . . . 34
4.2.1. Participants . . . 34
4.2.2. Tasks . . . 35
4.2.3. Study Procedure . . . 35
4.2.4. Pre-Study . . . 36
4.2.5. Main Study Procedure . . . 39
4.2.6. Post-Study Questionnaire . . . 40
4.3. Results . . . 41
4.3.1. Descriptive Overview . . . 41
4.3.2. Efficiency and Accuracy . . . 42
4.3.3. Usability and Workload . . . 43
4.3.4. Expertise Effects . . . 43
4.3.5. Task Success and Failure . . . 43
4.3.6. Baseline Comparison . . . 44

5. Discussion **47**
5.1. Interpretation of Results . . . 47
5.2. Limitations . . . 50

6. Conclusion **53**
6.1. Summary of Contributions . . . 53
6.2. Future Work . . . 54

A. Appendix **57**
A.1. User Interface . . . 57
A.2. Wrapper Execution Flows . . . 59
A.3. Execution Trace Examples . . . 63
A.4. Participant Assignment Details . . . 70
A.5. Task Descriptions . . . 72
A.6. Gold Answers . . . 72
A.7. User Study Results . . . 73

A.8. Execution Details of the AQL Interface 75
A.9. LangChain Execution Details 75
A.10. Reproducibility and Artifacts 76

List of Figures **77**

List of Tables **79**

Bibliography **81**

1. Introduction

1.1. Motivation

Data in modern organizations is distributed across a variety of heterogeneous systems, including relational databases, document repositories, email systems, and web-based resources such as Wikipedia or public APIs. Retrieving information from multiple such sources has become a common requirement in enterprise environments and knowledge work [1, 2].

Despite this availability, multi-source retrieval remains challenging. In practice, users often need to interact with multiple independent tools and interfaces to obtain the required information. This fragmented workflow increases cognitive effort, reduces efficiency, and complicates complex information-seeking tasks.

A representative example arises in administrative settings. The mobility department of the City of Munich processes citizen inquiries regarding traffic signal behavior at specific intersections. Answering such requests requires retrieving and relating information from multiple sources, including regulatory documents, geospatial data, and historical measurements. This involves multiple retrieval and interpretation steps, such as identifying applicable regulations based on spatial properties and validating them against recorded data. In practice, this process is largely manual, time-consuming, and requires domain expertise.

Existing approaches address parts of this problem. Traditional solutions, such as federated databases and data virtualization, provide unified access to distributed sources but require schema alignment and pre-integration. More recent approaches based on retrieval-augmented generation (RAG) and agent-based systems using large language models [3, 4] can automate retrieval steps but often lack transparency and limit user control.

There remains a need for approaches that enable transparent and interactive retrieval from heterogeneous data sources without requiring prior integration.

This thesis addresses this gap through the design of **Rubicon**, a query interface that supports explicit multi-step retrieval with user control over each step.

Unlike traditional data integration approaches, the system does not support unified query execution across multiple sources within a single query. Instead, multi-source tasks are expressed as sequences of explicit retrieval steps.

1.2. Problem Statement

Although numerous systems exist for querying individual data sources, supporting retrieval tasks that span multiple heterogeneous sources remains challenging. Existing approaches typically fall into two categories.

Traditional query systems provide precise control and transparency but are largely limited to structured data and require significant technical expertise. In contrast, natural-language interfaces and automated agent systems simplify access but often obscure the retrieval process, limiting transparency and user control.

This results in a fundamental tension between expressiveness, usability, and transparency in multi-source information retrieval. Users require systems that support flexible composition and iterative refinement of information from multiple sources while maintaining visibility into the retrieval process.

Heterogeneity of data sources Different data sources expose heterogeneous data models and interfaces. Relational databases provide structured tables accessible via SQL, whereas web APIs, email systems, and document collections often return semi-structured or unstructured data. Reconciling results across these modalities requires mechanisms for representation and transformation into compatible formats.

Fragmented multi-source workflows Cross-source information needs typically involve multiple retrieval steps. Users must query different systems, switch between tools and interfaces, and manually compose intermediate results. Existing tools rarely support expressing and managing such workflows in a unified and consistent way [5].

Lack of transparency in automated retrieval Automated systems based on agents or natural-language interfaces often obscure the sequence of retrieval operations. This limits users' ability to verify results, debug errors, and understand system behavior, making transparency and explainability essential [6].

Difficulty of query refinement When multi-source queries produce incomplete or incorrect results, users must inspect intermediate steps and iteratively adjust the retrieval process. This requires systems that expose execution steps and support controlled refinement.

1.3. Research Objectives and Questions

This thesis focuses on the design and implementation of **Rubicon**, and evaluates its effectiveness for transparent and interactive multi-source information retrieval. At its core, Rubicon models information retrieval as an explicit sequence of operations over individual data sources. Users formulate queries using a small set of composable operators (e.g., `FIND`, `FROM`, `WHERE`, `RETURN`, `?`), which are executed step by step across

sources. Each operation targets a single data source, while multi-source tasks are realized through the composition of multiple queries. This design emphasizes transparency, user control, and predictable execution behavior over automated cross-source integration.

The system is implemented using a modular architecture with source-specific wrappers and an execution engine that produces structured execution traces, enabling users to inspect and refine retrieval processes.

The approach is evaluated through a controlled user study comparing the operator-based interface with a manual workflow using conventional tools. The evaluation considers task completion time, accuracy, usability, cognitive workload, and the influence of user expertise.

The following research questions guide this work:

RQ1 (Expressiveness) Can a small set of composable query operators express typical multi-source information retrieval tasks through explicit, stepwise retrieval across heterogeneous data sources?

RQ2 (Efficiency) Does the proposed interface reduce task completion time compared to a manual workflow using separate tools?

RQ3 (Performance) Does the interface improve the correctness of results when solving multi-source retrieval tasks?

RQ4 (Usability and Workload) Does the interface improve perceived usability and reduce cognitive workload compared to manual workflows?

RQ5 (Accessibility and Skill Gap) Does the proposed approach reduce the dependency on user expertise when performing information retrieval tasks?

1.4. Contributions

This thesis makes the following contributions:

- The design of a user-facing, operator-based query model that enables explicit and structured retrieval from heterogeneous data sources.
- A transparent execution framework that exposes intermediate steps through structured execution traces.
- A hybrid retrieval architecture that combines deterministic query execution with controlled LLM-based translation and retrieval-based repair.
- The implementation of a prototype system, **Rubicon**, supporting multiple heterogeneous sources.
- A controlled user study evaluating efficiency, correctness, usability, and expertise dependency.
- Empirical insights into the trade-off between automation and user control in multi-source retrieval.

1.5. Thesis Structure

The remainder of this thesis is structured as follows.

Chapter 2 introduces the background concepts relevant to this work, including enterprise information retrieval, federated data systems, retrieval-augmented generation, agentic retrieval systems, and the role of large language models in information retrieval.

Chapter 3 describes the design of the proposed system. It introduces the overall system concept, the agentic query language, the query processing model, the retrieval architecture, and the transparency mechanisms that support inspection of intermediate retrieval steps.

Chapter 4 presents the evaluation of the proposed approach, including the user study design, the study procedure, and the empirical results.

Chapter 5 discusses the findings of the evaluation and reflects on the strengths and limitations of the proposed system.

Finally, Chapter 6 summarizes the main contributions and findings of this thesis and outlines directions for future work.

2. Background

2.1. Enterprise Information Retrieval

Enterprise data environments Compared to classic document retrieval settings, enterprise information retrieval occurs in more complex and diverse data environments. Relational databases, knowledge bases, file repositories, email systems, and web resources are among the autonomous systems found in enterprise environments where relevant data is dispersed. These sources lead to fragmented and partially inconsistent data landscapes due to differences in format, interfaces, schemas, and update frequency [7, 2].

Enterprise data spans both structured and unstructured representations [7]. Structured data is stored in databases with explicit schemas, whereas unstructured information appears in documents, emails, or web pages [8].

In addition, enterprise data is dynamic, as records are updated and new sources are introduced. This makes it difficult to maintain a comprehensive and stable index. Practical retrieval systems must therefore operate under conditions of incomplete uniformity and continuous change [2].

Multi-source information needs Enterprise information needs often extend beyond retrieving individual documents and instead require specific facts or related attributes derived from multiple sources. Tasks such as entity identification or accessing contextual information typically involve retrieving and composing data from databases, emails, internal documentation, and external resources. This differs from traditional search settings, where returning a ranked list of documents is often sufficient [7, 8].

Relevant information is rarely contained within a single system. Early work on distributed retrieval emphasized the need to query multiple collections rather than relying on a centralized index [9]. In enterprise environments, this challenge is further amplified by differences in modality, structure, and access semantics.

Enterprise queries often combine multiple types of intent. While Broder's taxonomy distinguishes navigational, informational, and transactional search [10], these intents frequently overlap in practice. Users may need to locate entities, inspect contextual information, and retrieve specific attributes within a single task.

While such tasks may resemble integrated querying across sources, they are often addressed through sequential retrieval and manual composition rather than unified query execution. This highlights the need for approaches that support explicit, stepwise multi-source retrieval.

Challenges of enterprise retrieval A central challenge of enterprise retrieval is that relevance cannot be determined at the document level alone. Because information is distributed across heterogeneous sources, retrieval involves not only identifying relevant content but also selecting appropriate sources, formulating source-specific queries, and reconciling results with different structures and semantics. This introduces challenges beyond classical information retrieval, including schema mismatch, inconsistent representations, and varying retrieval capabilities [7, 2].

Another challenge is the limited comparability of results across sources. Traditional retrieval models assume that documents can be ranked within a shared representation space. In enterprise settings, however, sources may return structured tuples, full-text documents, or semi-structured metadata, making unified ranking and result presentation difficult [8, 11].

Enterprise retrieval is also constrained by practical factors such as access restrictions, incomplete indexing, and the need for high precision [7]. These limitations highlight the need for approaches that explicitly account for heterogeneous sources and support structured and transparent interaction.

2.2. Federated Data Systems

Mediator-wrapper architecture Federated data systems provide unified access to distributed and heterogeneous data sources without requiring full physical integration. Instead of consolidating data into a single repository, they introduce a virtual integration layer that enables querying autonomous sources through a common interface [12].

A central concept is the mediator-wrapper model [13]. Wrappers encapsulate data sources and translate between local data models and a shared representation, allowing consistent access across multiple systems.

Mediators coordinate query execution by decomposing queries, delegating sub-queries to relevant sources, and composing the results into a single output. This separation of concerns supports modular system design and flexible integration of data sources.

The mediator-wrapper architecture decouples queries from source-specific details but relies on a shared representation that enables queries to be expressed independently of individual source schemas.

Schema integration A key requirement for federated data systems is the ability to reconcile differences between the schemas of individual data sources. Since sources are designed independently, schemas may differ in structure, naming conventions, data types, and levels of abstraction. Schema integration establishes correspondences between these different representations to enable cross-source queries [14].

A central challenge is schema matching, which involves identifying semantically equivalent or related elements across schemas, such as aligning attribute names and resolving structural differences. Automating this process remains difficult in complex real-world settings [14].

Query processing in federated systems relies on mappings between source schemas and a shared representation, requiring queries to be rewritten into source-specific forms [15]. Maintaining these mappings across evolving data sources introduces significant overhead, particularly in dynamic environments.

While schema integration enables join-like operations across structured sources, such operations depend on well-defined mappings and compatible schemas. These assumptions are difficult to satisfy in heterogeneous environments involving semi-structured or unstructured data.

Limitations of federated approaches While federated data systems provide a principled approach for accessing multiple data sources, they exhibit several limitations in enterprise environments. A primary limitation is the reliance on predefined schema mappings and a global representation. Establishing and maintaining these mappings requires substantial manual effort and domain expertise, particularly as the number and diversity of sources increase [14, 15].

Another limitation is the assumption that data sources can be described using structured schemas. Many enterprise sources, such as documents, emails, or web content, are only partially structured or unstructured, making federated systems less suitable for such data.

Federated approaches also face challenges in query optimization across autonomous sources. Since data remains distributed and under the control of individual systems, mediators have limited knowledge of source capabilities and data characteristics, which can lead to inefficient queries or incomplete results [12].

In addition, federated systems provide limited support for interactive and exploratory query workflows, as queries are typically formulated against a predefined global schema.

As a result, federated approaches are less suited for scenarios that require flexible, stepwise retrieval and composition of information across different sources without explicit schema alignment.

2.3. Retrieval-Augmented Generation

RAG architecture Retrieval-Augmented Generation (RAG) combines information retrieval with large language models to improve performance on knowledge-intensive tasks. By incorporating external information retrieved at query time, RAG enables access to up-to-date, domain-specific knowledge without retraining [3].

A RAG system typically consists of two main components: a retriever and a generator. Given a query, the retriever selects relevant documents or passages from an external corpus, which are then used by the generator to produce a response conditioned on both the query and the retrieved information [3, 16].

RAG follows a modular design in which retrieval and generation are loosely coupled, allowing different retrieval strategies and language models to be combined. As a result,

it has been widely adopted in applications such as open-domain question answering, knowledge assistants, and document-grounded dialogue systems [17].

Retrieval stage The retrieval stage selects relevant information from external knowledge sources based on the user query. In modern RAG systems, this is typically achieved using dense retrieval methods, where queries and documents are mapped into a shared representation space using neural embedding models [18].

Dense retrieval enables the identification of semantically relevant information beyond exact term matching, allowing the system to retrieve relevant content even when lexical overlap is limited [18].

The quality of the retrieval stage directly influences the final output, as the generator relies on the retrieved context to produce accurate responses [16].

Generation stage In the generation stage, the language model produces an output conditioned on the user query and the retrieved documents. By incorporating external context, the model can ground its responses in retrieved evidence rather than relying solely on internal representations [3].

The quality of the generated output depends on the relevance and completeness of the retrieved context, as missing or irrelevant information can lead to incomplete or misleading responses [17].

Limitations of RAG systems Despite their effectiveness, RAG systems exhibit several limitations in multi-source information retrieval. A key limitation is the lack of transparency in the interaction between retrieval and generation. While retrieved documents may be accessible, their use during generation is not directly observable, making it difficult to understand or verify how responses are produced [16].

RAG systems are also susceptible to hallucinations. If the retrieved context is incomplete, ambiguous, or irrelevant, the model may generate content that is not supported by the underlying data [19].

In addition, RAG systems are primarily designed for text-based retrieval and generation, limiting their applicability in environments with structured data sources such as relational databases. They do not provide explicit mechanisms for specifying structured constraints or controlling the retrieval and generation process.

As largely automated pipelines, RAG systems offer limited user control and make iterative refinement difficult. As a result, they do not fully support explicit and interpretable multi-source retrieval, particularly in scenarios that require structured composition and user-guided execution.

2.4. Agentic Retrieval Systems

Tool-using language models Recent advances in large language models enable systems that not only generate text but also interact with external tools, such as invoking APIs

or retrieving external information as part of their reasoning process [20].

The ReAct framework combines reasoning and acting by interleaving intermediate reasoning steps with tool use, enabling iterative problem solving and multi-step retrieval [4]. Similarly, approaches such as Toolformer allow models to learn when and how to use external tools within generated sequences [21].

Agentic retrieval pipelines Agentic retrieval systems organize model interactions into multi-step pipelines that iteratively retrieve information, process intermediate results, and refine actions. These pipelines typically follow cycles of reasoning, action, and observation, enabling dynamic adaptation based on retrieved information [20].

Such approaches are particularly useful for complex tasks where relevant information is distributed across sources or cannot be fully specified in advance. However, this flexibility introduces additional complexity in execution and control.

Planning and tool orchestration A central aspect of agentic systems is planning and tool orchestration. Given a task, the model determines which actions to execute, in which order, and how to combine results, resulting in dynamic and non-predefined workflows [20].

This requires handling heterogeneous interfaces and data formats, as different tools return structured, semi-structured, or unstructured outputs, increasing the complexity of integrating retrieved information.

Limitations of autonomous agents Despite their flexibility, agentic retrieval systems exhibit several limitations. A primary challenge is the lack of transparency, as execution logic is generated dynamically and is difficult to predict or verify [20].

Agent behavior is inherently variable, meaning that identical queries may lead to different execution paths and results. Errors in early steps can propagate through subsequent actions, reducing reliability.

User control is limited, as high-level goals are translated into implicit action sequences. This makes targeted intervention and iterative refinement difficult and can lead to inefficiencies such as redundant tool calls.

As a result, agentic retrieval systems do not fully support structured, reproducible, and user-controlled multi-source retrieval, motivating approaches that provide explicit guidance and observable execution processes.

2.5. Large Language Models in Information Retrieval

Capabilities of large language models Large language models (LLMs) provide strong capabilities in natural language understanding and generation, enabling a wide range of tasks without task-specific training. Models such as GPT-3 demonstrate that LLMs can generalize across tasks to some extent through few-shot prompting [22]. LLMs

can interpret complex inputs and generate context-aware responses that go beyond surface-level text matching.

In information retrieval, these capabilities enable semantic interpretation of queries, inference of implicit user intent, and support for tasks such as query reformulation, summarization, and information composition [16].

LLMs for query interpretation LLMs enable the translation of natural language queries into structured representations or semantically enriched search expressions, reducing the need for explicit query formulation.

They extend neural retrieval approaches such as BERT [23] by supporting flexible transformations, including mapping natural language to structured query elements and extracting relevant constraints or entities. This makes them particularly useful in environments where different data sources require different query formats.

Cost and performance considerations The use of LLMs introduces high cost and performance challenges. Inference requires substantial computational resources, resulting in higher latency and operational cost compared to traditional retrieval methods [16]. This can affect the responsiveness of interactive systems, particularly in scenarios requiring rapid query refinement.

As a result, many systems restrict LLM usage to tasks where their added value is highest, such as semantic interpretation or query transformation, while relying on more efficient retrieval components for large-scale document processing.

Limitations of LLM-based retrieval LLMs exhibit several limitations in structured and multi-source retrieval settings. Their probabilistic generation process can lead to inconsistent outputs, reducing reproducibility and reliability. In addition, their internal reasoning processes are not directly observable, making verification and debugging difficult.

LLMs also lack mechanisms for enforcing structured constraints or for integrating heterogeneous data sources in a controlled, deterministic manner. As a result, they do not fully support transparent, controllable, and structured multi-source retrieval, motivating hybrid approaches that combine LLM capabilities with explicit and observable execution frameworks.

2.6. Human-in-the-Loop Information Retrieval

Interactive retrieval systems Human-in-the-loop information retrieval emphasizes the active role of users in the retrieval process. Interactive systems support iterative exploration, enabling users to refine queries, inspect intermediate results, and adjust their strategy as their understanding of the information need evolves [24].

In such settings, information needs are often not fully specified at the outset but develop progressively during interaction. This makes iterative refinement particularly important for complex or exploratory search tasks [24].

In multi-source environments, interaction becomes even more important. Users must interpret partial results from different systems and adapt their queries accordingly, requiring continuous user involvement and flexible adjustment throughout the process.

These characteristics highlight the need for systems that support structured query formulation and interactive, transparent execution across heterogeneous data sources.

Transparency and explainability Transparency and explainability are essential components of human-in-the-loop systems. Users must be able to understand how results are produced in order to assess their correctness and relevance, including insight into system behavior and the transformation of inputs into outputs [6].

This requires systems to make intermediate steps observable and their behavior predictable, enabling users to detect errors, identify mismatches between intent and system interpretation, and develop an accurate mental model of system behavior [6].

In complex or high-stakes scenarios, interpretability becomes critical, requiring systems to expose not only final outputs but also the underlying processing steps [25].

User control in retrieval processes User control is a central aspect of human-in-the-loop retrieval. Users should be able to influence query execution, source selection, and the combination of results [6]. This enables targeted intervention when outputs are incomplete, incorrect, or misaligned with the user's intent.

Traditional retrieval systems often provide limited control, while fully automated systems make many decisions implicitly. Both extremes pose challenges, either restricting expressiveness or limiting the ability to adjust system behavior.

Human-in-the-loop approaches address this by combining system support with explicit user guidance, allowing users to influence key aspects of the retrieval process while the system assists with execution.

In multi-source retrieval, user control becomes particularly important due to heterogeneous data sources and complex query workflows. Explicit query formulation and stepwise execution enable users to maintain oversight and adapt their strategy, supporting reliable and interpretable retrieval.

2.7. Related Systems and Frameworks

LangChain LangChain is an open-source framework for building applications that combine large language models with external tools, retrieval components, and agent-based execution [26]. It provides abstractions for integrating model providers, vector stores, APIs, and custom tools, enabling the construction of retrieval-augmented and agentic pipelines.

The framework primarily targets developers by offering programmatic control over retrieval and tool orchestration. Query execution is defined directly in code, allowing flexible composition of multi-step workflows and custom retrieval logic.

Despite this flexibility, LangChain does not provide a structured query language or an explicit user-facing representation of the retrieval process. Execution remains implicit, and transparency depends on developer-defined tracing mechanisms. As a result, support for explicit, user-guided query formulation and reproducible execution is limited.

OpenClaw OpenClaw is an open-source, self-hosted framework for building agent-based systems that connect language models with tools, memory components, and external services [27]. It provides an integrated execution environment with support for persistent memory, multi-step interactions, and extensible tool definitions.

Tool usage is defined through structured configurations, enabling the invocation of external services within agent-driven workflows. OpenClaw follows an agent-driven interaction model in which users specify high-level goals, while execution is determined by the system.

As a result, the retrieval process remains implicit, limiting user control over query structure and execution steps and reducing suitability for explicit and interpretable multi-source retrieval.

Other tool-based retrieval frameworks Frameworks such as Haystack and LlamaIndex provide abstractions for combining retrieval components, language models, and external tools into pipelines [28, 29]. Haystack focuses on pipeline-based architectures with explicitly defined processing flows, while LlamaIndex emphasizes document ingestion, indexing, and retrieval across structured and unstructured data.

These frameworks primarily target developers and require programmatic specification of workflows. They do not provide an explicit user-facing query language or a transparent execution model, limiting user control and reproducibility in multi-source retrieval.

Existing systems either emphasize automated retrieval or developer-defined workflows but do not provide a user-facing, structured query abstraction that enables transparent and interactive multi-source retrieval through explicit stepwise execution.

3. System Design

3.1. System Overview

This chapter presents the design of **Rubicon**, including its architectural components, query language, and execution model.

3.1.1. Design Goals

Transparency Rubicon is designed to make query execution explicitly observable. For each query, the system returns both the final result and a structured execution trace that records the main processing steps.

The trace captures the transformation from user input to executable operations across all components. In addition, the user interface exposes and highlights the structural query elements `FIND`, `FROM`, `WHERE`, `RETURN`, and `?`, allowing users to directly relate query structure to execution behavior.

Composability Rubicon supports composable query construction through a structured query language consisting of `FIND`, `FROM`, and optional `WHERE` and `RETURN` clauses.

This structure enables incremental refinement by allowing individual clauses to be extended or modified without affecting the overall query structure.

Support for Heterogeneous Sources Rubicon provides unified access to heterogeneous data sources, including relational databases, email systems, Wikipedia, and an LLM-based source. These sources differ in structure, access mechanisms, and query capabilities.

All sources are accessed through a uniform query language and interaction model. The target source is explicitly specified in the `FROM` clause using predefined identifiers such as `db`, `email`, `wiki`, or `llm`.

Each query is executed against a single source by design. Cross-source queries and relational join semantics across sources are not supported. Multi-source workflows are instead expressed as sequences of separate queries.

Abstraction Layer Rubicon introduces a source-independent abstraction layer that separates query formulation from execution.

User queries are transformed into a structured internal representation that captures their logical components independently of any specific data source. This representation serves as the interface between query processing and execution.

Each data source is encapsulated by a wrapper that maps the abstract representation to source-specific operations. Despite differences in underlying data models, all wrappers return results in a unified format together with execution traces.

User Control Rubicon enforces explicit user control over query formulation and execution.

Users specify the data source, selection criteria, and output format directly within the query. Default behaviors, such as implicit return types, remain overridable.

This design ensures that execution behavior is fully determined by the query structure rather than implicit system decisions.

3.1.2. System Components

Rubicon follows a modular architecture in which components are assigned distinct functional responsibilities. These components separate query formulation, processing, source access, and result presentation.

User Interface The system provides a web-based interface for query formulation and result inspection. It serves as the primary interaction layer, allowing users to submit queries and receive both results and execution traces.

The interface presents the execution trace alongside the result, enabling inspection of how queries are interpreted and processed.

In addition, an integrated information panel provides structured access to available data sources, attributes, and supported query constructs. Screenshots of the interface are provided in Appendix A.1.

The interface does not perform any processing logic and remains strictly separated from the execution layer.

Agentic Query Language (AQL) Queries are expressed using the Agentic Query Language (AQL), which defines a structured interface for specifying retrieval tasks. It serves as the formal interface between user input and the internal processing components of Rubicon. A detailed description is provided in Section 3.2.

Query Processing Engine The query processing engine transforms AQL queries into a structured representation suitable for execution. It interprets query structure, resolves the referenced data source, and prepares the information required for downstream components.

This component forms the boundary between the query language and the execution layer, enabling a consistent, source-independent processing model.

Retrieval Architecture The retrieval architecture provides access to data sources through a wrapper-based abstraction. Each source is encapsulated by a dedicated wrapper that implements a common execution interface.

This design separates source-independent query processing from source-specific execution. Wrappers map abstract query representations to concrete source operations.

The retrieval architecture thereby constitutes the execution layer of the system, bridging query processing and source interaction.

Execution Trace The execution trace records the sequence of processing steps performed during query execution. It is attached to each result and captures how a query is interpreted and executed across system components.

The trace includes intermediate transformations, source-specific operations, and model-assisted steps, providing a structured view of the execution process.

Figure 3.1 illustrates the system architecture and the corresponding query processing flow.

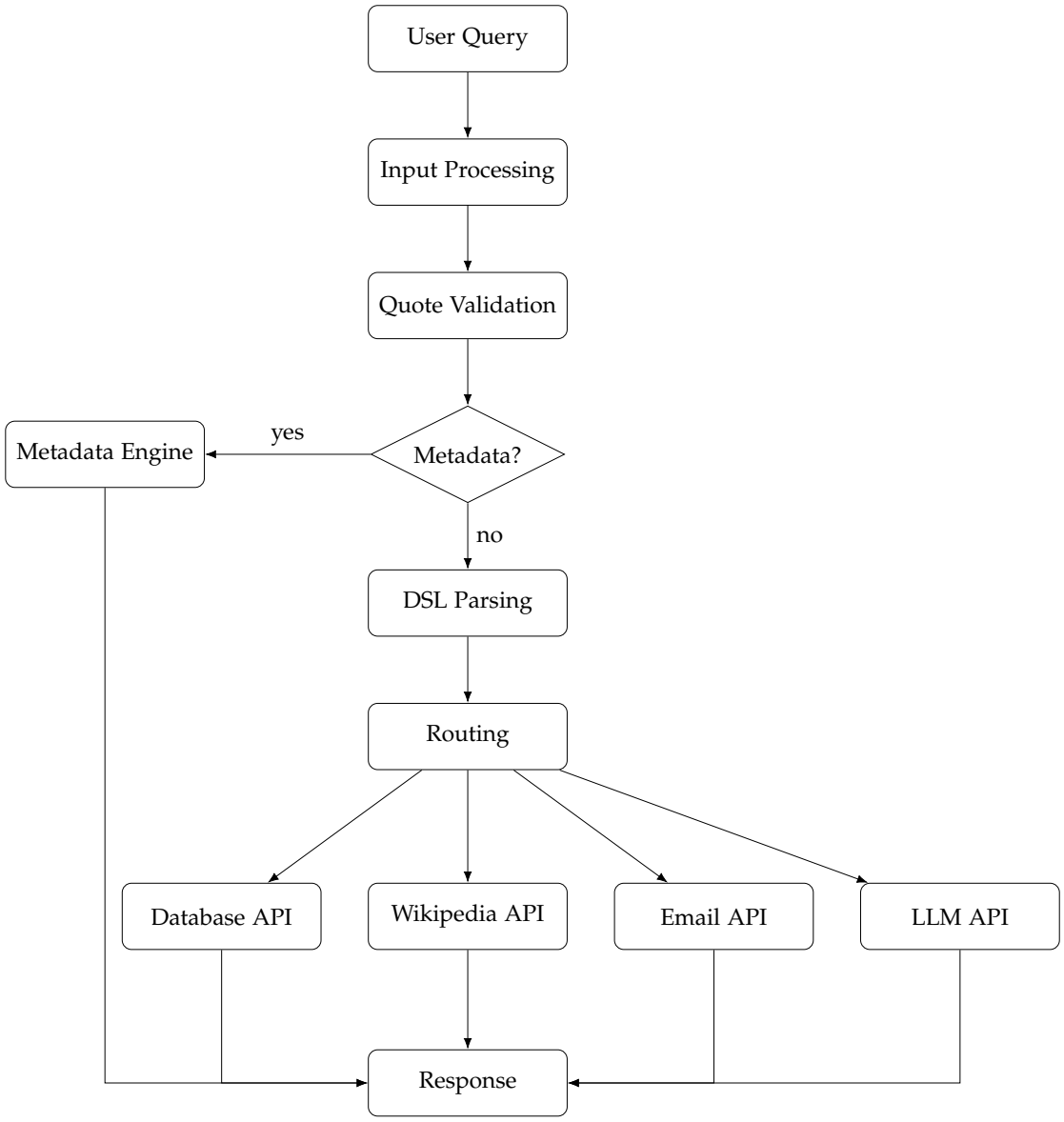


Figure 3.1.: System architecture and query processing flow, from user input to result generation.

3.1.3. End-to-End Query Flow

This section outlines the lifecycle of a query from user input to result generation. The process follows a structured pipeline, illustrated in Figure 3.1.

Query Formulation Queries are entered through the user interface using AQL, consisting of the clauses `FIND`, `FROM`, and optional `WHERE` and `RETURN`. The input is normalized and validated before further processing.

Metadata queries using the `?` operator are handled separately and bypass the standard execution pipeline.

Query Interpretation The validated query is transformed into a structured internal representation that captures its logical components and resolves the target data source.

Retrieval and Processing The query is dispatched to the corresponding source wrapper, which maps the abstract representation to source-specific operations and executes the query.

Execution may include deterministic processing as well as controlled semantic translation or refinement when required.

Result Presentation The result is mapped to a unified output format based on the specified or inferred return type. In addition, an execution trace is attached, capturing the main processing steps.

3.2. Agentic Query Language (AQL)

The Agentic Query Language (AQL) is a domain-specific query language (DSL) [30] for expressing retrieval tasks over heterogeneous data sources. It provides a structured syntax that enables queries to be specified in a consistent and source-independent manner.

AQL consists of a fixed set of clauses that define the retrieval target, the data source, optional filtering conditions, and the output format.

3.2.1. Language Design and Concepts

AQL is based on a small set of clauses with clearly defined roles. This structure separates query formulation from execution and provides a uniform representation of retrieval tasks.

Operators and Structure Queries are composed of the clauses `FIND`, `FROM`, `WHERE`, and `RETURN`. Each clause corresponds to a logical operator:

- `FROM`: defines the data source and establishes the execution context,
- `WHERE`: specifies filtering conditions over the source,
- `FIND`: defines the information to be retrieved,
- `RETURN`: determines the output representation.

This structure reflects a decomposition into source selection, restriction, and projection. Although expressed as a single statement, a query is interpreted as a sequence of logical operations over a source-specific execution context.

3.2.2. Query Structure

This section defines the semantics of the AQL clauses. Each clause contributes a specific component to the query and determines how the retrieval task is specified.

FIND The `FIND` clause specifies the information to be retrieved.

In structured sources, it corresponds to a projection over explicitly defined attributes. In semi-structured or unstructured sources, it defines the primary retrieval target, such as an entity, topic, or search term.

The clause may also contain semantically specified expressions, typically enclosed in single quotation marks. These expressions represent natural language descriptions of the desired output and require interpretation during query processing.

FROM The `FROM` clause specifies the data source on which the query is executed and establishes the execution context.

Sources are referenced using predefined identifiers such as `db`, `email`, `wiki`, or `llm`.

WHERE The optional `WHERE` clause specifies conditions that restrict the result set.

Conditions may be expressed either as structured constraints or as semantically specified expressions. Structured constraints correspond to directly interpretable predicates over attributes or fields. Semantically specified expressions, typically enclosed in single quotation marks, represent natural language conditions that require interpretation during query processing.

In structured sources, conditions are mapped to deterministic filtering operations. In less structured sources, they may be interpreted using semantic matching or translation mechanisms.

RETURN The optional `RETURN` clause specifies the representation of the query result.

Supported return types include structured formats, such as tables, and textual representations. If no `RETURN` clause is provided, a default representation is selected based on the data source and result structure.

Metadata Operator The `?` operator retrieves metadata about a data source or its structure, such as available attributes or supported query capabilities.

Metadata queries are evaluated independently of standard queries and do not require the full set of clauses.

3.2.3. Example Queries

Simple Queries The following examples illustrate basic AQL usage across different sources:

```
find identifier from db.ZIP_USA
find Germany from wiki
find email.subject from email
find climate change from llm
db ?
```

These queries demonstrate the use of the `FIND` and `FROM` clauses as well as the metadata operator.

Complex Queries More advanced queries extend this structure by combining multiple clauses and more expressive conditions:

```
find 'city name and state name', state_abr, city_type
from db.ZIP_USA
where STATE_ABBR = "PR" and not CITY_TYPE = "D"
return table

find wiki.title, wiki.pageid, wiki.url
from wiki
where "MIT" in wiki.abstract and not "press" in wiki.title

find email.subject, email.from
from email
where email.hasAttachments = "true" and email.from = "google"

find privacy
from email
where 'security warning from google in the last 30 days' return text

db.city_name in ZIP_USA ?
```

These examples illustrate the combination of projection, filtering, and output specification within a single query. They further show how both structured constraints and semantically specified expressions can be used within the same query.

Expressions enclosed in single quotation marks (e.g., `'...'`) denote semantically specified inputs. Such expressions may occur in both `FIND` and `WHERE` clauses and require interpretation during query processing.

AQL defines the structure and semantics of queries independently of their execution. The following section introduces the processing model that maps this structure to executable operations.

3.3. Query Processing

This section describes the processing model that transforms AQL queries into executable operations and results.

The model separates syntactic interpretation, source-independent query representation, execution planning, and result construction. It provides a uniform processing pipeline while allowing source-specific execution strategies.

Query processing follows a structured sequence of transformations. In addition to deterministic processing, the model incorporates probabilistic components in a controlled manner. LLMs are used exclusively for bounded semantic translation tasks, while retrieval-based methods are applied for repair, refinement, and semantic matching when deterministic execution is insufficient.

3.3.1. Query Representation and Parsing

Structured Query Representation The structured query representation captures the contents of the AQL clauses `FIND`, `FROM`, `WHERE`, and `RETURN` in a source-independent form. It also includes derived elements such as the resolved source identifier, optional subsource information, and the normalized return type.

A key design decision is to preserve semantically specified expressions. Expressions enclosed in single quotation marks are retained verbatim and not interpreted during parsing. This allows subsequent stages to distinguish between deterministic query fragments and those requiring semantic interpretation, while keeping the parser source-agnostic.

The representation serves both as an explicit encoding of query structure and as the interface for downstream execution components.

Syntax Interpretation Parsing is performed using a rule-based procedure that identifies the syntactic structure of the query without inferring semantic intent.

Quotation marks are normalized to ensure a consistent textual representation. During parsing, the content of single-quoted segments is temporarily masked to prevent reserved keywords such as `FROM`, `WHERE`, or `RETURN` from being misinterpreted as clause delimiters.

The parser then identifies clause boundaries and extracts their contents from the original input, preserving the exact user query while enabling reliable structural interpretation.

Double-quoted expressions are treated as deterministic literals, whereas single-quoted expressions are retained as semantic markers for subsequent processing. This distinction enables the coexistence of symbolic filtering and semantic interpretation within the same query.

Operator Identification Based on the structured representation, the query is interpreted in terms of logical operators corresponding to the AQL clauses.

At this stage, operators are identified independently of any source-specific execution. This abstraction defines the logical roles required for subsequent execution planning.

3.3.2. Execution Model

The execution model defines how structured queries are transformed into executable operations. It separates logical query intent from source-specific execution while maintaining a uniform processing structure.

Operator Abstraction Execution is modeled as a sequence of transformations over intermediate states. Logical operators correspond to source selection, filtering, retrieval or projection, and output transformation.

These operators are defined independently of the underlying data source and are instantiated through source-specific execution strategies. This abstraction enables a uniform execution model across heterogeneous systems.

Query Plan Construction Based on the structured representation, an implicit query plan is constructed that defines the required execution stages.

Execution follows a fixed logical order aligned with the structure of AQL. The plan may include deterministic stages, semantic translation stages, and repair stages.

Semantic translation is applied when semantically specified expressions require interpretation. Repair stages are triggered when execution fails or yields unsatisfactory results and are implemented using retrieval-based methods.

Operator Sequencing Operators are executed in a predefined sequence reflecting their logical dependencies.

Execution begins with source resolution and validation, followed by filtering (`WHERE`), retrieval or projection (`FIND`), and output transformation (`RETURN`). This ordering reduces ambiguity and ensures traceable execution.

Source Access The query plan is executed through source-specific wrappers that map abstract operators to concrete operations.

Each wrapper implements the required execution strategy for its underlying system while adhering to the shared operator abstraction.

Intermediate Result Handling Execution proceeds through intermediate states that capture successive transformations of the query and its results.

Outputs produced by probabilistic components are treated as intermediate artifacts unless the selected source is the LLM itself. Such outputs are validated and integrated into subsequent deterministic steps.

Intermediate states form the basis of the execution trace by recording transformations such as translation, normalization, and repair.

3.3.3. Result Construction

Result Combination After execution, intermediate states are transformed into a unified result representation.

Since each query is executed against a single source, result combination does not involve cross-source integration. Instead, it consolidates the outcomes of the execution process into a structured output aligned with the query.

Depending on the execution path, this may involve structuring retrieved data, aggregating intermediate results, or post-processing generated outputs.

Outputs produced by probabilistic components are retained as part of the execution process rather than treated as opaque results. This preserves their role within the overall execution and allows subsequent validation or transformation if required.

Output Formatting The combined result is mapped to a user-visible format, either specified explicitly or inferred by default.

Structured results are represented as tables, while textual outputs are presented as concise responses or summaries. Return types are validated against source capabilities, with defaults applied when necessary.

In addition to the primary result, an execution trace is appended, capturing the sequence of processing steps.

3.4. Retrieval Architecture

This section describes the retrieval architecture of Rubicon, focusing on source-specific wrappers that instantiate the execution model for individual data sources.

The system follows a wrapper-based design in which each data source is encapsulated by a dedicated module implementing a shared interface.

3.4.1. Source Wrappers

Wrapper Concept Each data source is accessed through a dedicated wrapper that instantiates the abstract query representation for execution.

This design separates source-independent query processing from source-specific execution logic and follows the principle of mediator-based information systems [13].

New data sources can be integrated by implementing additional wrappers without modifying the core processing pipeline.

Each wrapper maps logical query operators to source-specific operations, executes the query, and returns results in a unified format together with an execution trace.

Unified Interface All wrappers implement a common interface that accepts the structured query representation and returns a result with an execution trace.

This interface abstracts from differences in data models and access mechanisms, while ensuring a consistent execution pattern across sources.

3.4.2. Wrapper Implementations

Rubicon supports four types of data sources: relational databases, email systems, Wikipedia, and an LLM-based source. Each wrapper implements a source-specific execution strategy while adhering to the shared interface.

The following paragraphs describe the execution logic of each wrapper, including operator mapping, retrieval, and, where applicable, semantic translation and repair.

Relational Database The relational database schema is derived from the BEAVER benchmark, providing a realistic enterprise data structure.

The database wrapper translates the structured query representation into SQL and executes it against a relational database. Execution is primarily deterministic, with optional semantic translation and retrieval-based repair.

Execution begins with validation of the parsed query and resolution of the target table. An SQL statement is constructed using a rule-based mapping from AQL clauses to SQL components.

If semantically specified expressions are present, an LLM is used to translate them into SQL fragments, such as column selections or predicates. This translation is constrained by the database schema to ensure syntactic validity.

The resulting SQL statement is executed. If execution fails due to schema-related errors, a retrieval-based repair step is applied by matching invalid identifiers to schema elements using embedding-based similarity. The corrected query is then re-executed, forming an iterative execution-repair loop.

If no suitable repair can be identified, the system returns an error response.

Empty results do not trigger repair, as they indicate a valid but unsatisfied query.

Finally, results are formatted according to the requested return type, typically as structured tables or textual outputs.

Figure 3.2 summarizes the main execution stages of the database wrapper. A detailed execution flow of the relational database wrapper is provided in Appendix A.2.

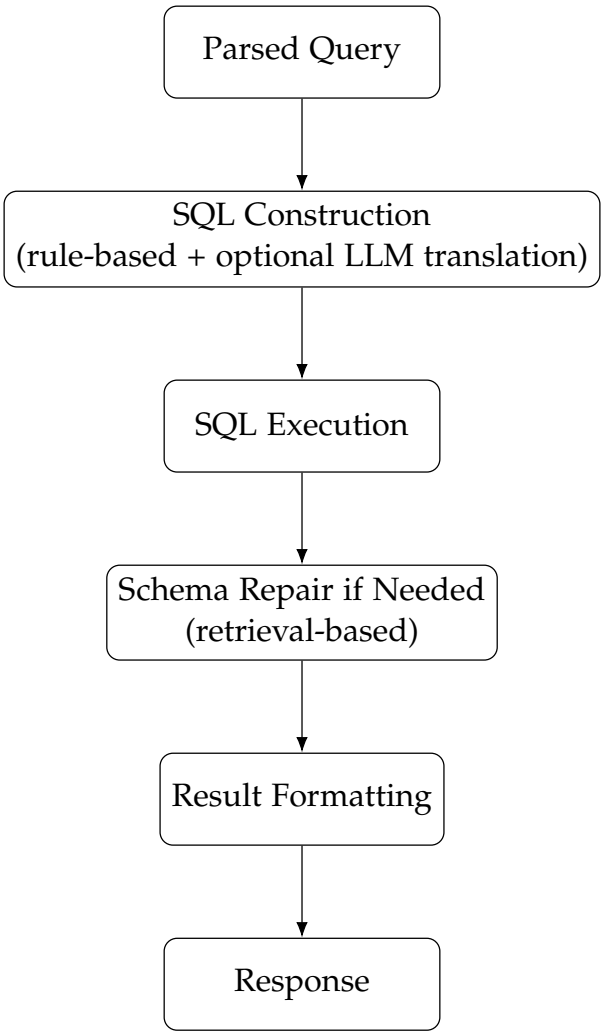


Figure 3.2.: Simplified execution flow of the database wrapper, showing SQL construction, execution, optional repair, and result formatting.

Wikipedia The Wikipedia wrapper provides access to semi-structured knowledge and supports two execution modes: attribute-based search and direct page retrieval.

In attribute-based mode, search queries are derived from the `FIND` or `WHERE` clause to retrieve candidate pages. These are filtered and evaluated based on explicitly extracted attributes such as `title`, `pageid`, `url`, and `abstract`.

In direct retrieval mode, the `FIND` clause is interpreted as an entity lookup. The title is normalized and, if necessary, resolved using an LLM for semantically specified inputs. The corresponding page is then retrieved, with disambiguation handled through iterative resolution.

Once a page is retrieved, the wrapper supports multiple subsources, including summary extraction, table extraction via HTML parsing, and image retrieval. Results are formatted according to the selected subsource and return type.

To ensure bounded execution, the number of search iterations is limited.

Figure 3.3 summarizes the main execution modes of the Wikipedia wrapper. A detailed execution flow is provided in Appendix A.2.

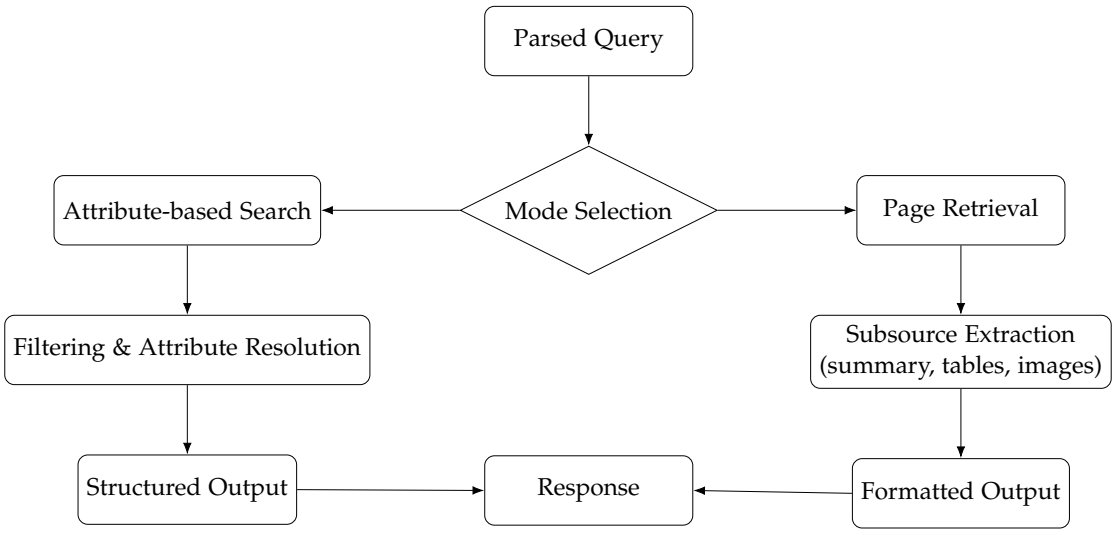


Figure 3.3.: Simplified execution flow of the Wikipedia wrapper, illustrating attribute-based search and direct page retrieval modes.

Email System The email wrapper implements a multi-stage retrieval pipeline combining structured query processing with semantic translation and refinement.

If a `WHERE` clause is present, it is transformed into a structured logical representation, including normalization of boolean expressions. Semantically specified expressions are translated into Gmail search queries using an LLM.

The constructed query is executed against the email system to retrieve candidate messages. If no results are returned, a retrieval-based repair mechanism is applied by refining the query using embedding-based similarity over message content and metadata.

If necessary, a relaxation step broadens query constraints before a final retrieval attempt.

After retrieval, additional filtering may be applied based on message attributes and content.

The `FIND` clause is then applied, supporting attribute projection, keyword-based selection, or semantic ranking. If textual output is requested, an LLM may be used for summarization.

Figure 3.4 summarizes the main execution stages of the email wrapper. A detailed execution flow is provided in Appendix A.2.

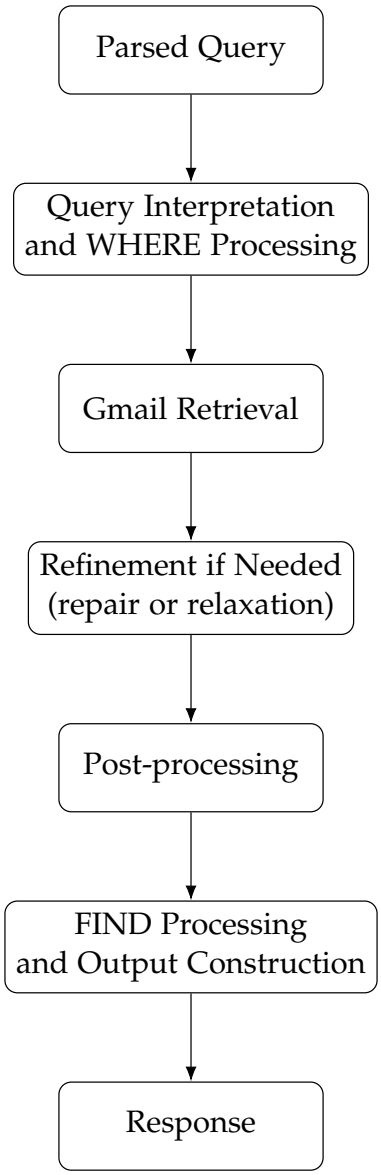


Figure 3.4.: Simplified execution flow of the email wrapper, showing query interpretation, retrieval, refinement, post-processing, and result construction.

LLM-Based Source The LLM-based wrapper provides generative text processing through structured prompt construction.

Execution begins by deriving system and user prompts from the structured query representation. Prompt templates depend on the requested return type and differ for textual and structured outputs.

The language model is then invoked to generate a response. If the request fails, an error is returned.

For structured outputs, the model is instructed to produce data in a predefined format, which is parsed and rendered as a table. Textual outputs are returned directly after formatting.

In contrast to other wrappers, this component does not perform retrieval or repair, as the language model itself serves as the source of information.

Figure 3.5 summarizes the execution stages of the LLM-based wrapper. A detailed execution flow is provided in Appendix A.2.

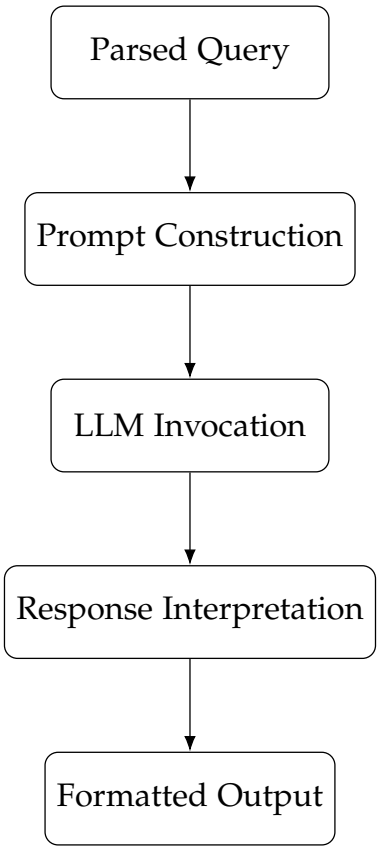


Figure 3.5.: Simplified execution flow of the LLM-based wrapper, showing prompt construction, model invocation, and response interpretation.

3.5. Transparency and Execution Trace

Transparency is a central design principle of Rubicon and is realized through a structured execution trace that exposes internal processing steps.

The execution trace records how a query is interpreted, transformed, and executed across all stages.

By making these steps explicit, the system enables inspection and verification of query execution.

3.5.1. Execution Trace Model

Trace Representation The execution trace is represented as an ordered sequence of structured entries, each corresponding to a stage in the execution pipeline.

Each entry records the performed operation together with the associated intermediate state, such as parsed query components, constructed queries, intermediate results, model inputs and outputs, or repair decisions.

The representation is consistent across data sources, enabling interpretation of execution flows independent of source-specific details.

Trace Examples Example execution traces are provided in Appendix A.3.

These examples illustrate how deterministic operations and model-based transformations are captured within a unified structure.

Recording Execution Steps Execution steps are recorded incrementally throughout the processing pipeline. Each component contributes entries to the trace, including parsing, routing, source-specific execution, and repair.

Deterministic operations are recorded explicitly, while probabilistic operations include their inputs and outputs. Repair steps record both candidate selection and applied corrections, including unsuccessful attempts.

This design follows principles of interpretable machine learning, particularly regarding the transparency of model-assisted components [25, 31].

3.5.2. Comparison to Black-Box Retrieval Systems

Limitations of Black-Box Retrieval Many modern retrieval systems, particularly those based on LLMs, operate as black boxes, producing results without exposing intermediate processing steps.

This lack of transparency limits the ability to understand, verify, and debug system behavior. Errors such as hallucinations or incorrect interpretations cannot be attributed to specific processing stages, making systematic analysis difficult. Additionally, the absence of explicit execution steps restricts iterative query refinement.

These limitations are well documented for LLM-based systems, where outputs may appear plausible despite lacking grounding in factual or structured data [32].

In multi-source settings, such issues may propagate across sources and lead to inconsistent results.

Benefits of Transparent Execution Rubicon addresses these limitations by exposing execution through a structured trace.

This enables inspection of individual processing steps and allows users to distinguish between deterministic operations, model-assisted transformations, and repair mechanisms. As a result, query behavior becomes verifiable and can be systematically refined.

By making execution explicit, the system avoids the opacity of end-to-end retrieval approaches.

3.6. Design Rationale

This section motivates the central design decisions underlying the system architecture.

Rubicon follows a hybrid design that combines deterministic processing with controlled machine-learning-based and retrieval-based components. This design aims to balance flexibility, reliability, and transparency while preserving explicit user control and interpretability in multi-source query processing.

While inspired by agentic concepts, the system restricts autonomy in favor of explicit and controllable execution.

3.6.1. Limitations of Fully Agentic Retrieval

Lack of Transparency Fully agentic retrieval systems rely on LLMs to interpret user intent, select tools, and execute actions autonomously. While this enables flexible interaction, it obscures intermediate decisions such as tool selection, query formulation, and reasoning steps.

As a result, system behavior becomes difficult to inspect or debug, and errors cannot be attributed to specific stages of execution.

In contrast, Rubicon enforces an explicit query structure and exposes execution steps, enabling inspectable and reproducible query processing.

Unpredictable Tool Usage Agentic systems, such as those based on reasoning–acting paradigms [4], dynamically determine tool usage and parameterization based on model outputs.

This can lead to inconsistent execution, as identical queries may result in different strategies. Incorrect tool selection or parameter generation may further produce invalid or misleading results.

The system addresses this by requiring explicit source selection through the `FROM` clause and by following predefined execution paths. Tool usage is therefore controlled and consistent.

Why Dual-Stage Retrieval The system separates semantic interpretation from execution correction through a dual-stage retrieval approach.

Purely LLM-based approaches support flexible query formulation but suffer from hallucinations, lack of grounding, and limited reproducibility [32]. Purely deterministic approaches ensure correctness but are less robust to ambiguous or natural language expressions.

To address this, the system assigns distinct roles to different components. LLMs are used exclusively for semantic translation, mapping natural language expressions to structured query elements. Retrieval-based methods are applied for repair and refinement after initial execution.

This separation ensures flexible interpretation while maintaining grounded and controlled execution.

3.6.2. Model Selection and Retrieval Strategy

Choice of Language Model The system employs the model `gpt-4o-mini` for semantic translation tasks.

The model is used exclusively for constrained mapping tasks, such as translating natural language expressions into structured query elements. This restricted role reduces the need for larger models and allows prioritization of low latency, cost efficiency, and sufficient semantic accuracy.

Larger models would increase computational cost without substantial benefits for these bounded tasks, while smaller models may lack the reliability required for consistent translation.

To limit the impact of non-determinism, model outputs are treated as intermediate representations and integrated into subsequent deterministic execution steps.

Choice of Embedding Model For retrieval-based repair, the system uses the sentence embedding model `all-MiniLM-L6-v2`, based on Sentence-BERT [33].

Sentence-BERT enables efficient semantic similarity computation by mapping text to a dense vector space, where cosine similarity can be used to identify relevant candidates.

The model is selected for its strong performance and low computational overhead. Given the relatively small candidate spaces in the system, high-capacity embedding models are not required.

This choice enables efficient similarity computation while maintaining sufficient accuracy for real-time query processing.

Retrieval-Augmented Repair Strategy The system adopts a retrieval-based approach for error recovery and query refinement.

Instead of generating corrections via a language model, candidate elements are retrieved from the data context, such as schema attributes or observed values, and the most semantically similar option is selected. Corrections are applied only if a predefined similarity threshold is exceeded.

This approach differs from retrieval-augmented generation (RAG) [3], where retrieved context is used to guide content generation. In contrast, retrieval is used here to constrain and validate corrections.

As a result, modifications remain grounded in valid system entities.

3.6.3. Why Operator-Based Control

Explicit Query Structure Rubicon enforces an operator-based query structure through AQL.

Queries consist of clearly defined clauses corresponding to logical operations such as source selection, filtering, and projection. This structure makes query intent explicit and provides a direct mapping to the execution model.

In contrast to free-form natural language interfaces, this reduces ambiguity and enables consistent interpretation.

User-Guided Retrieval By requiring explicit specification of the data source and query structure, the system preserves user control over retrieval.

This avoids implicit decisions about source selection and execution strategy, as found in agentic systems. Instead, execution is fully determined by the query structure.

This is particularly relevant in multi-source settings, where explicit control ensures consistent execution.

3.6.4. Why Transparent Execution

Inspectability Rubicon exposes execution through a structured trace, making intermediate processing steps observable.

This enables analysis of query execution and identification of potential errors, particularly in hybrid settings involving both deterministic and model-based components.

Iterative Query Refinement Transparency enables iterative refinement by allowing users to adjust queries based on observed execution behavior.

This establishes a feedback-driven interaction model in which queries are progressively improved through inspection of execution steps.

The impact of these design decisions is evaluated in Chapter 4.

4. Evaluation

4.1. Evaluation Goals

The evaluation focuses on Rubicon, the system implementing the AQL interface. For readability, we refer to it as AQL when emphasizing the user-facing query interface.

Study Objectives The goal of the evaluation is to assess whether the proposed AQL interface improves the performance and accessibility of enterprise information-seeking and data-related tasks compared to a conventional manual workflow.

In particular, the study investigates whether the AQL interface reduces the dependency on prior technical expertise when performing tasks that require retrieving information from heterogeneous data sources.

The evaluation focuses on overall task performance, task efficiency, and robustness, as well as the extent to which the system reduces the performance gap between users with different levels of technical expertise.

Hypotheses The study investigates the following hypotheses.

Main effects

- **H1 (Efficiency):** Task completion time is lower when using AQL compared to the manual workflow.
- **H2 (Performance):** Task performance (measured by F1-score) is higher when using AQL compared to the manual workflow.
- **H3 (Usability and Workload):** The AQL interface yields higher perceived usability and lower perceived workload compared to the manual workflow.

Skill-gap and accessibility hypotheses

- **H4 (Skill Gap Reduction):** The performance difference between low-expertise and high-expertise users is expected to be smaller in the AQL condition than in the manual condition.
  - **H4a (Accessibility Contrast):** Low-expertise participants using AQL are expected to perform at least as well as, or not substantially worse than, high-expertise participants using the manual workflow.
  - **H4b (Low-Expertise Benefit):** Low-expertise participants are expected to perform better with AQL than with the manual workflow.

4.2. User Study Design

The evaluation was conducted as a controlled user study comparing two approaches for performing information retrieval tasks: the proposed AQL interface and a conventional manual workflow.

Participants were assigned to one of two experimental conditions:

- **AQL condition**: participants completed tasks using the AQL interface.
- **Manual condition**: participants completed the same tasks using conventional tools such as database interfaces, email search, Wikipedia search, or ChatGPT.

The study follows a **between-subjects design**, in which each participant interacted with only one interface condition. This design avoids learning and transfer effects that could occur if participants used multiple systems sequentially [34, 35].

Each participant completed multiple tasks to increase measurement reliability and reduce the influence of individual task variability [34].

This design enables a direct comparison of both approaches under comparable task conditions. Figure 4.1 illustrates the study design and participant distribution.

	Interface	
	AQL	**Manual**
Higher Expertise	$n = 3$ (initial)	$n = 3$ (initial)
Lower Expertise	$n = 3$	$n = 3$

Figure 4.1.: Conceptual study design with interface condition as the between-subjects factor and participant expertise used for balanced group assignment and subsequent analysis.

4.2.1. Participants

A total of **12 participants** were initially assigned to the study. However, 2 participants (P7 and P11) did not complete the main task session and were excluded from the analysis.

The final analysis is therefore based on **10 participants**.

Both excluded participants belonged to the higher-expertise groups, with one participant in each interface condition (AQL and Manual). As a result, the balance between conditions is preserved, and no systematic imbalance between conditions is observed.

The participant pool included students, researchers, and professionals from both STEM and non-STEM backgrounds, reflecting a broad range of technical expertise.

Participant expertise was estimated using a pre-study survey described in Section 4.2.4.

4.2.2. Tasks

Task Design Participants completed information retrieval tasks that required combining information from heterogeneous data sources. The tasks were designed to reflect realistic enterprise data-work scenarios in which users must access multiple systems and tools to answer information needs [36, 37].

Each task required participants to retrieve specific information and provide a final answer in textual form. The answers were evaluated against gold-standard solutions described in Appendix A.6.

Task Categories Each participant completed **8 tasks** in total. The full task descriptions are provided in Appendix A.5.

The tasks were divided into four categories:

- 2 LLM tasks
- 2 database tasks
- 2 email tasks
- 2 Wikipedia tasks

Task difficulty was controlled by designing tasks within each category to be comparable in complexity and required reasoning effort.

4.2.3. Study Procedure

Experimental Setup The study was conducted in a controlled setting. Participants first completed a pre-study survey collecting demographic and background information. They then completed the assigned task set using either the AQL interface or the manual workflow, depending on their condition. No additional training was provided to reflect first-time usage scenarios. After completing the tasks, participants filled out a post-study questionnaire assessing usability and perceived workload.

Design and Randomization Participants were assigned to one interface condition using stratified randomization based on expertise. Within each expertise stratum, participants were randomly assigned to either the AQL or the manual condition, ensuring a balanced expertise distribution across conditions [34].

Task order was fully randomized for each participant across all categories and presented in an interleaved sequence. This reduces potential order effects such as learning and fatigue [34].

Independent Variables The primary independent variable was the interface condition (AQL vs. Manual). Participant expertise was included as a continuous moderator variable, and task category (LLM, database, email, Wikipedia) as a control variable.

Dependent Variables The dependent variables correspond to the collected performance and subjective measures. Performance measures include task completion time, task performance (measured by F1-score), and dead end rate.

4.2.4. Pre-Study

Before the main study, a pre-study survey was conducted to collect participant background information and estimate technical expertise for balanced assignment to the experimental conditions.

Purpose The primary purpose of the pre-study was to assess participants' technical expertise, which was used for stratified assignment across conditions. This is relevant as the study examines whether the AQL interface reduces the dependency on prior technical expertise.

In addition, the pre-study collected descriptive background variables to characterize the participant sample and support the interpretation of the results.

Procedure The pre-study survey collected the following variables:

- current role
- educational background
- SQL proficiency (1–5 scale)
- SQL experience in years
- programming proficiency (1–5 scale)
- familiarity with advanced email search operators (1–5 scale)
- confidence in structuring prompts for ChatGPT (1–5 scale)
- Wikipedia usage frequency
- previously used tools
- typical data-related tasks performed by the participant

For the construction of the expertise score, only **SQL proficiency** and **programming proficiency** were used, as they directly reflect core technical capabilities relevant for interacting with data systems.

SQL experience in years was retained as a descriptive variable and considered in a robustness analysis but excluded from the main score to avoid overemphasizing SQL-related indicators. All other variables were used for descriptive analysis of the participant sample.

Expertise Score Construction Participant expertise was operationalized as a combined score based on SQL proficiency and programming proficiency, both measured on a five-point Likert scale (1 = no experience, 5 = very confident).

To ensure equal contribution, both variables were standardized using z-scores:

$$z(x_i) = \frac{x_i - \bar{x}}{s} \tag{4.1}$$

where x_i is the value of participant i, $\bar{x}$ is the mean of the variable, and s is the standard deviation.

The expertise score was then calculated as:

$$ExpertiseScore_i = z(SQL_i) + z(Programming_i) \tag{4.2}$$

This formulation treats SQL and programming proficiency as equally weighted components of technical expertise.

Descriptive Statistics Table 4.1 summarizes the descriptive statistics of the variables used for expertise score construction. The results indicate variability in both SQL and programming proficiency across participants.

Table 4.1.: Descriptive statistics of the variables used for expertise score construction.

Variable	Mean	Standard Deviation
SQL Proficiency	2.42	1.68
Programming Proficiency	2.92	1.78

Robustness Check As a robustness check, an alternative expertise score was computed by additionally including self-reported SQL experience in years:

$$ExpertiseScore_i^{alt} = z(SQL_i) + z(Programming_i) + z(SQLYears_i) \tag{4.3}$$

Including SQL experience does not change the rank ordering of participants, and the resulting stratification into higher- and lower-expertise groups remains identical.

The alternative score is highly consistent with the original score (Pearson $r \approx 0.99$, Spearman $\rho = 1.00$), indicating a stable operationalization of expertise.

The expertise balance between conditions remains comparable (AQL: 0.45, Manual: -0.45) relative to the original specification (AQL: 0.28, Manual: -0.28).

Overall, these results confirm that the expertise grouping is robust to the inclusion of SQL experience.

Expertise Ranking & Participant Assignment The resulting expertise scores ranged from -1.92 to 2.71, indicating substantial variation in technical expertise within the participant pool. Negative values represent below-average expertise, while positive values indicate above-average expertise relative to the sample.

Participants were ranked based on their expertise score and divided into two equally sized strata. The six highest-ranked participants formed the *higher-expertise* group, and the six lowest-ranked participants formed the *lower-expertise* group.

The resulting expertise groups were:

- **Higher Expertise** ($n = 6$): P8, P11, P10, P12, P7, P1
- **Lower Expertise** ($n = 6$): P9, P2, P3, P4, P5, P6

Two participants (P1 and P9) were located close to the threshold between groups. A ranking-based split was used instead of a fixed threshold to ensure transparency and equal group sizes.

Within each expertise stratum, participants were randomly assigned to one of the two experimental conditions, resulting in four balanced groups:

- AQL – Higher Expertise: P8, P10, P7
- Manual – Higher Expertise: P11, P12, P1
- AQL – Lower Expertise: P9, P3, P5
- Manual – Lower Expertise: P2, P4, P6

Each condition therefore included three higher-expertise and three lower-expertise participants.

To verify the balance between conditions, the mean expertise score was computed for each condition (Table 4.2). The results are centered around zero (AQL: 0.28, Manual: -0.28), indicating comparable expertise distributions.

Table 4.2.: Expertise distribution across experimental conditions.

Condition	Participants	Mean Expertise Score
AQL	P8, P10, P7, P9, P3, P5	0.28
Manual	P11, P12, P1, P2, P4, P6	-0.28

The final participant assignment and additional details are provided in Appendix A.4.

Two participants (P7 and P11) did not complete the main study and were excluded from the final analysis. The expertise ranking and assignment procedure remain unchanged, as they are based on the pre-study.

4.2.5. Main Study Procedure

Following the pre-study, participants completed the main task session in their assigned interface condition. Each participant completed 8 tasks across the four task categories described above. Task performance and user interactions (e.g., completion time, task outcome, and task attempts) were logged automatically.

Measures The study collected both objective and subjective measures.

Objective measures included:

- Task completion time per task
- Task performance (measured by F1-score)
- Dead end rate (proportion of tasks without a correct solution, e.g., due to timeouts or unresolved queries)

Subjective measures included:

- Perceived usability (SUS-based items)
- Perceived workload (NASA-TLX-based items)

Task Performance Evaluation Task performance was evaluated using a combined approach for structured (tabular) outputs and natural language responses (see Appendix A.6).

Tabular Outputs For structured outputs (database, email, and Wikipedia tasks), correctness was evaluated at the row level. Let G denote the gold-standard result set and A the participant's result set.

- **True Positive (TP):** A predicted row matching a row in G, or manually verified as correct
- **False Positive (FP):** A predicted row not matching any row in G and not verifiable as correct
- **False Negative (FN):** A row in G missing in A

Precision, recall, and F1-score were computed based on these counts. True negatives are not defined, as the set of non-relevant rows is not explicitly enumerated.

Natural Language Outputs For tasks requiring natural language responses, correctness was evaluated using predefined key points.

- **True Positive (TP):** A correct key point is present
- **False Negative (FN):** A required key point is missing
- **False Positive (FP):** An incorrect or hallucinated statement is present

Precision, recall, and F1-score were computed accordingly.

Hallucination Handling Hallucinations, including incorrect statements in text or invalid rows in structured outputs, were counted as false positives.

Aggregation F1-scores were computed per task and aggregated at the participant level across all tasks.

4.2.6. Post-Study Questionnaire

After completing the task session, participants filled out a post-study questionnaire assessing usability, workload, and subjective task experience.

Questionnaire Design The questionnaire consisted of three components:

- **Usability (SUS-based items):** A subset of items inspired by the System Usability Scale (SUS) [38] capturing ease of use, learnability, and intended frequency of use.
- **Workload (NASA-TLX-based items):** A reduced version of the NASA Task Load Index (NASA-TLX) [39], including the dimensions mental demand, temporal demand, and effort.
- **Perceived effectiveness and expertise dependence:** Additional Likert-scale items assessing expressiveness, required technical knowledge, and user confidence.

All items were measured on a five-point Likert scale (1 = strongly disagree / very low, 5 = strongly agree / very high).

Qualitative Feedback In addition to the quantitative items, participants provided open-ended feedback on:

- positive aspects,
- sources of confusion or frustration, and
- suggestions for improvement.

These responses provide qualitative context for interpreting the quantitative results and highlight usability issues.

Aggregation and Analysis For the quantitative analysis, responses were aggregated at the participant level. Group- and condition-level statistics were computed for comparison between the AQL and manual conditions.

No inferential statistical tests were conducted due to the limited sample size. Results are therefore interpreted as descriptive trends rather than statistically significant effects. While the results are interpreted descriptively, the observed differences between conditions are substantial across multiple metrics.

4.3. Results

4.3.1. Descriptive Overview

Aggregated Results Table 4.3 and Table 4.4 summarize performance across groups and interface conditions.

Across conditions, AQL outperforms the manual workflow across all measured metrics. In particular, AQL achieves higher task performance (F1: 0.68 vs. 0.44), higher precision (0.74 vs. 0.61), and higher recall (0.65 vs. 0.41), while also exhibiting a lower dead end rate (0.23 vs. 0.30). Completion times are slightly lower for AQL (431s vs. 448s).

The observed differences are consistent across multiple metrics, including performance, recall, and robustness, which strengthens confidence in the observed trends.

At the group level, performance differences depend on user expertise. The improvement is most pronounced for lower-expertise participants, where AQL substantially outperforms the manual workflow (F1: 0.76 vs. 0.41). Differences for higher-expertise participants are smaller (0.55 vs. 0.50).

Per-participant results are provided in Appendix A.7 (Table A.3).

Table 4.3.: Aggregated performance metrics by group.

Group	Avg Time (s)	Avg F1	Avg Precision	Avg Recall	Dead End Rate
AQL-High	300	0.55	0.57	0.53	0.38
AQL-Low	519	0.76	0.86	0.72	0.13
Manual-High	556	0.50	0.56	0.50	0.25
Manual-Low	375	0.41	0.64	0.35	0.33

Table 4.4.: Aggregated performance by interface condition.

Condition	Avg Time (s)	Avg F1	Avg Precision	Avg Recall	Dead End Rate
AQL	431	0.68	0.74	0.65	0.23
Manual	448	0.44	0.61	0.41	0.30

Subjective ratings are summarized in Table 4.5 and Table 4.6. AQL is associated with higher perceived usability and effectiveness, and lower workload compared to the manual workflow.

Table 4.5.: Perceived usability, workload, and effectiveness by group (mean and SD).

Group	Usability	SD	Workload	SD	Effectiveness	SD
AQL-High	3.00	0.94	2.50	2.12	2.50	0.71
AQL-Low	3.22	0.77	3.44	0.69	4.00	0.87
Manual-High	2.67	0.47	3.67	0.94	2.25	0.35
Manual-Low	1.89	0.38	4.33	0.00	2.17	0.58

Table 4.6.: Perceived usability, workload, and effectiveness by interface condition (mean and SD).

Condition	Usability	SD	Workload	SD	Effectiveness	SD
AQL	3.13	0.73	3.07	1.28	3.40	1.08
Manual	2.20	0.56	4.07	0.60	2.20	0.45

4.3.2. Efficiency and Accuracy

Task Completion Time (H1) AQL achieves lower average completion times than the manual workflow (431s vs. 448s).

At the group level, this effect depends on user expertise. Higher-expertise participants complete tasks substantially faster with AQL (300s vs. 556s), whereas lower-expertise participants require more time (519s vs. 375s).

This indicates that AQL improves efficiency for experienced users, while less experienced users incur additional interaction overhead.

Overall, H1 is partially supported, with efficiency gains depending on user expertise.

Task Performance (H2) AQL substantially improves overall task performance compared to the manual workflow (F1: 0.68 vs. 0.44).

This improvement is consistent with higher precision (0.74 vs. 0.61) and recall (0.65 vs. 0.41), indicating more accurate and more complete results.

At the group level, the performance gain is most pronounced for lower-expertise participants (0.76 vs. 0.41), while differences for higher-expertise participants are smaller (0.55 vs. 0.50).

Overall, these results support H2, indicating a substantial and practically relevant performance improvement.

4.3.3. Usability and Workload

Perceived Usability and Workload (H3) AQL is perceived as substantially more usable than the manual workflow (3.13 vs. 2.20), with lower workload (3.07 vs. 4.07) and higher perceived effectiveness (3.40 vs. 2.20).

Variability is generally low for usability (SD = 0.73 for AQL, 0.56 for manual), while workload shows higher variability in the AQL condition (SD = 1.28), indicating more diverse user experiences.

At the group level, differences depend on user expertise. For higher-expertise participants, AQL improves usability (3.00 vs. 2.67) and reduces workload (2.50 vs. 3.67), while perceived effectiveness remains similar (2.50 vs. 2.25).

For lower-expertise participants, the differences are more pronounced. The manual workflow is associated with very low usability (1.89) and high workload (4.33), whereas AQL improves usability (3.22) and reduces workload (3.44). Perceived effectiveness is highest for lower-expertise participants using AQL (4.00).

The zero variance in workload for Manual-Low (SD = 0.00) indicates consistently high workload in this group.

Overall, these results support H3, indicating improved usability, reduced workload, and higher perceived effectiveness for AQL, particularly for lower-expertise participants.

4.3.4. Expertise Effects

Skill Gap Reduction (H4) In the manual condition, a performance gap exists between higher- and lower-expertise participants (F1: 0.50 vs. 0.41).

In contrast, this gap is substantially reduced and reversed in the AQL condition, where lower-expertise participants outperform higher-expertise participants (0.76 vs. 0.55).

These results indicate that AQL reduces reliance on prior technical expertise and enables less experienced users to achieve high performance.

Low-Expertise Performance (H4a, H4b) Lower-expertise participants using AQL (F1: 0.76) outperform higher-expertise participants using the manual workflow (F1: 0.50), supporting H4a.

Furthermore, lower-expertise participants achieve substantially higher performance with AQL than with the manual workflow (0.76 vs. 0.41), supporting H4b.

Per-participant results are provided in Appendix A.7 (Table A.3).

4.3.5. Task Success and Failure

Dead End Rate AQL reduces the overall dead end rate compared to the manual workflow (0.23 vs. 0.30).

At the group level, this effect is particularly pronounced for lower-expertise participants (0.13 vs. 0.33), indicating fewer failed or incomplete tasks in this group.

These results highlight a trade-off between efficiency and robustness: while manual workflows can be faster in some cases, they more frequently lead to unsuccessful task outcomes, whereas AQL improves task success and reliability.

4.3.6. Baseline Comparison

Evaluation Scope The user study evaluates human performance and usability effects of AQL compared to a manual workflow. In contrast, the LangChain comparison focuses on system-level behavior, assessing execution efficiency, accuracy, and computational cost independent of user interaction. These perspectives complement each other by covering both human-centered and system-level performance.

LangChain Baseline In addition to the manual workflow, a LangChain-based agent was evaluated as a system-level baseline representing an LLM-driven retrieval approach. The agent used a lightweight GPT-based model (GPT-5-mini) as the underlying language model. All queries from the user study were executed using the LangChain agent to ensure comparability.

Execution Models The LangChain agent follows a ReAct-style strategy with iterative tool use and model-driven planning. In contrast, AQL executes structured queries deterministically, with optional LLM support for specific subtasks.

This distinction enables a comparison between fully LLM-driven orchestration and a structured, operator-based execution model with controlled LLM integration.

Evaluation Metrics The LangChain baseline was evaluated using the same queries as the user study. For each query, we measured accuracy, execution time, tool usage, and token usage as a proxy for computational cost.

Accuracy is defined as binary task success (correct vs. incorrect), focusing on end-to-end outcomes rather than partial correctness.

For AQL, token usage is zero for deterministic queries and limited to specific subtasks when LLM support is used.

Results Overview Figure 4.2 shows a comparison between AQL and LangChain in terms of accuracy and execution time.

AQL produces correct results for all evaluated queries in this setting (8/8), while LangChain achieves an accuracy of 0.75 (6/8).

AQL also exhibits lower execution times (8s vs. 49s on average) and limited token usage, while LangChain shows higher runtime and greater variability in computational cost.

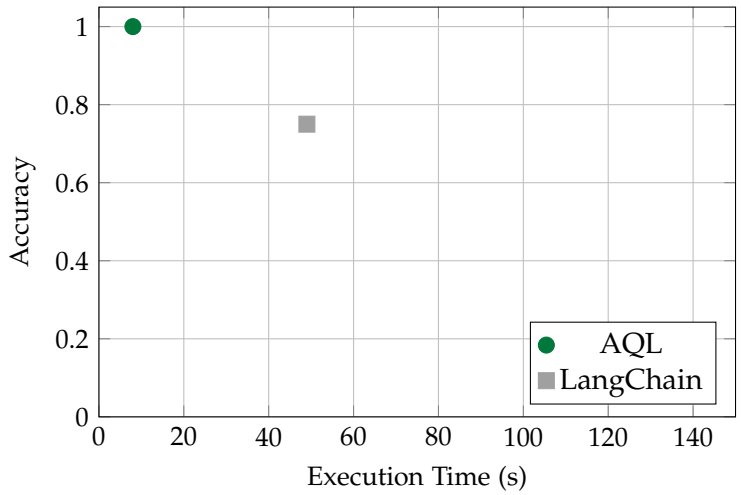


Figure 4.2.: Accuracy versus execution time for AQL and LangChain. Points represent average values across all queries.

Accuracy LangChain does not produce correct results for queries Q1 and Q4. In Q1, execution is not completed, while in Q4 no valid result is returned. In contrast, AQL returns correct results for all queries within the defined system constraints.

Efficiency AQL achieves lower and more consistent execution times, with an average runtime of approximately 8 seconds. LangChain shows higher and more variable execution times, with an average of approximately 49 seconds and peak values exceeding 140 seconds. Detailed execution statistics are provided in Appendix A.8 and Appendix A.9.

Cost and Token Usage LangChain incurs higher computational cost due to iterative reasoning and repeated tool usage, resulting in increased token consumption (see Appendix A.9). In contrast, AQL limits LLM usage to specific subtasks and can operate without token cost in deterministic mode.

System Constraints Rubicon applies controlled constraints on certain data sources, such as limiting Wikipedia search iterations and database result sizes. These constraints may restrict completeness in some cases but ensure predictable execution time and resource usage. All returned results remain correct within the evaluated conditions.

All artifacts required to reproduce the evaluation are publicly available (see Appendix A.10).

5. Discussion

5.1. Interpretation of Results

Core Performance and Robustness The results indicate that Rubicon improves task outcomes primarily by increasing retrieval completeness and robustness. Across the evaluated tasks, participants using Rubicon achieved higher overall task performance than those using the manual workflow.

This advantage is reflected in both precision and recall, indicating that users produced fewer incorrect or hallucinated results while retrieving a larger share of relevant information. This suggests that the query model supports more systematic retrieval across heterogeneous sources than a fragmented manual workflow.

The lower dead end rate further indicates improved task robustness. Participants using Rubicon were more likely to reach a usable solution even in more complex tasks, whereas manual workflows more frequently led to incomplete or unsuccessful outcomes. A likely explanation is that Rubicon makes intermediate retrieval steps explicit, thereby making them easier to inspect and refine.

These findings suggest that the main benefit of Rubicon lies not only in faster access to information, but also in more reliable task completion and higher-quality results, thereby supporting **RQ3** by demonstrating improved correctness and completeness in multi-source retrieval tasks.

Efficiency and Interaction Cost While Rubicon improves correctness and robustness, its effect on efficiency is more nuanced. Overall completion times are similar across conditions, but the impact differs depending on user expertise.

Higher-expertise participants complete tasks substantially faster with Rubicon, suggesting that the structured query model aligns well with existing mental models of data access and query formulation. For these users, explicit query composition reduces the need for manual coordination across sources. In contrast, lower-expertise participants require more time when using Rubicon. This can be attributed to the additional effort required to formulate queries and understand the interaction model, particularly in the absence of prior experience. Instead of relying on familiar search strategies or trial-and-error exploration, users must adopt a more structured approach to retrieval.

These findings point to a shift from cognitive effort to structured interaction effort: manual workflows require effort for information discovery and coordination, whereas Rubicon transfers this effort to explicit query formulation. As a result, efficiency gains depend on whether users can effectively leverage the structured interaction model.

Taken together, Rubicon does not universally reduce task completion time, but redistributes effort within the retrieval process, trading reduced cognitive overhead for increased interaction structure, thereby providing partial support for **RQ2**, as efficiency gains depend on user expertise and familiarity with the structured interaction model.

Accessibility and Skill Gap Reduction One of the most prominent findings of the evaluation is the substantial reduction of the performance gap between higher- and lower-expertise participants. In the AQL condition, lower-expertise users achieve performance levels comparable to, and in some cases exceeding, those of higher-expertise users, indicating a marked increase in accessibility.

This effect suggests that the operator-based query model reduces reliance on prior technical expertise by providing a consistent interaction paradigm. Instead of requiring knowledge of multiple tools, query languages, or data access strategies, users can express retrieval steps within a unified framework.

Lower-expertise participants report a substantially reduced need for advanced technical knowledge when using AQL compared to the manual workflow, indicating a lower perceived barrier to entry. In contrast, higher-expertise participants show only minor differences between conditions, suggesting that they are less dependent on interface support.

At the same time, the results indicate that AQL does not eliminate all expertise-related effects. Lower-expertise participants still report relatively high time pressure, reflecting the initial effort required to formulate structured queries and understand the interaction model. This suggests that AQL shifts the role of expertise rather than removing it entirely.

This is further supported by subjective ratings, where lower-expertise participants report a substantially lower need for technical knowledge in the AQL condition compared to the manual workflow (see Appendix A.4).

Together, these findings indicate that AQL reduces reliance on prior technical expertise by structuring interaction complexity into a more accessible form, enabling less experienced users to successfully perform multi-source retrieval tasks, thereby supporting **RQ5**.

Usability and Cognitive Load The results on usability and workload reflect a fundamental difference in how interaction complexity is distributed between the two approaches.

In the manual workflow, participants report substantial effort related to information discovery and coordination across multiple systems. Identifying relevant data sources and navigating between tools requires continuous context switching and imposes a high cognitive load. These difficulties are consistent with the observed high workload and low usability ratings in the manual condition.

In contrast, AQL reduces the need for cross-system navigation by providing a unified interaction model. Participants report that data sources are easier to access once the

query model is understood. This explains the overall improvement in perceived usability and the reduction in workload.

At the same time, AQL introduces a different form of interaction complexity. Users must explicitly formulate queries and understand the system's capabilities, which creates an initial learning barrier. Reported issues such as unclear system behavior, insufficient guidance, and uninformative error messages (as reflected in the qualitative feedback collected in the post-study questionnaire) suggest this phase may involve trial-and-error interaction.

Overall, effort shifts from exploratory search and tool coordination toward explicit query formulation within a consistent framework. As a result, cognitive load is reduced once the interaction model is internalized, while initial workload may remain elevated due to the learning curve.

This is further reflected in subjective ratings of temporal demand, which remain relatively high for lower-expertise participants in both conditions, although slightly reduced for AQL (see Appendix A.5).

This supports **RQ4**, indicating improved usability and reduced cognitive workload compared to manual workflows.

Transparency vs. Automation The comparison with the LangChain baseline highlights a fundamental design trade-off between structured, user-controlled retrieval and fully automated, LLM-driven orchestration.

Rubicon separates query formulation from execution and restricts LLM usage to well-defined subtasks. This results in a more predictable and inspectable execution process, where intermediate steps remain explicit.

In contrast, LLM-based approaches such as RAG and agent-based systems (e.g., LangChain) rely on iterative or retrieval-augmented reasoning, in which planning, tool selection, and intermediate steps are dynamically generated. This increases automation but introduces variability in execution time and computational cost, as well as a higher risk of incomplete or incorrect results due to error propagation across reasoning steps.

The observed differences in runtime, accuracy, and token usage reflect these underlying design choices rather than isolated performance characteristics. Structured query execution yields stable and resource-efficient behavior, whereas LLM-driven orchestration prioritizes flexibility and autonomy at the cost of predictability.

This reflects a broader trade-off: Rubicon requires explicit query construction, increasing transparency, control, and reproducibility. In contrast, agent-based approaches shift complexity to execution-time reasoning within the model, reducing user effort but obscuring the retrieval process.

Overall, the results suggest that structured and agent-based approaches are not direct replacements, but represent complementary design strategies with different strengths depending on the requirements for transparency, control, and automation.

In addition, the results indicate that the operator-based query model is sufficiently expressive to represent the evaluated multi-source retrieval tasks through explicit stepwise

composition. This is further supported by user feedback indicating that participants were able to clearly and precisely express their information needs, with particularly high ratings among lower-expertise users in the AQL condition (see Appendix A.6), thereby supporting **RQ1**.

Practical Implications The findings suggest that structured, operator-based query interfaces are particularly well-suited for real-world scenarios that require combining information from multiple heterogeneous sources.

In such settings, retrieval tasks involve multiple intermediate steps and domain-specific knowledge. By supporting explicit, stepwise retrieval within a unified interaction model, Rubicon can reduce coordination effort and improve the reliability of task outcomes.

At the same time, the results highlight that practical applicability depends on the balance between transparency, user effort, and system constraints. While structured execution enables predictable and controllable behavior, it may require additional interaction effort and can limit completeness under constrained retrieval settings.

The approach is therefore most beneficial in contexts where correctness, traceability, and controlled execution are prioritized over minimal interaction time or fully automated retrieval.

More broadly, these findings suggest that future multi-source retrieval systems may benefit from combining structured, transparent query mechanisms with selective automation, rather than relying solely on fully autonomous LLM-driven approaches.

5.2. Limitations

Study Limitations Several limitations should be considered when interpreting the results.

First, the sample size is relatively small ($n = 10$), limiting the generalizability of the findings and precluding inferential statistical testing. The results should therefore be interpreted as indicative rather than conclusive.

Second, parts of the evaluation setup were finalized relatively late in the study process, which limited the scope of more fine-grained analysis procedures. While descriptive statistics and standard performance metrics (e.g., F1-score, precision, recall, dead end rate) were computed, more advanced analyses such as systematic calibration, parameter tuning, or sensitivity analysis were not conducted. For example, aspects such as confidence thresholds or result filtering strategies were not explicitly explored. As a result, certain characteristics of system behavior could not be analyzed in greater depth.

Third, the study was conducted in a controlled setting with a fixed set of tasks. While designed to reflect realistic scenarios, these tasks cannot fully capture the diversity and complexity of real-world data-related environments.

Fourth, participants did not receive extensive training prior to using the systems. Although this reflects first-time usage, it may disadvantage approaches that require an

initial learning phase, such as AQL. Performance differences may therefore change with increased familiarity.

Finally, the manual condition represents a specific configuration of commonly used tools. Alternative tool combinations or user strategies may lead to different outcomes and are not covered in this study.

System Limitations Rubicon applies controlled constraints on certain data sources to ensure predictable execution behavior. For example, Wikipedia queries are limited to a fixed number of search iterations, and database queries restrict the number of returned rows. While these constraints improve stability and efficiency, they may lead to incomplete results in scenarios requiring exhaustive retrieval.

The system follows a single-source execution model, where each query operates on one data source and multi-source tasks are decomposed into sequential steps. This design improves transparency and simplifies execution, but introduces a trade-off between expressiveness and control. In particular, the lack of native cross-source joins limits the ability to formulate more complex queries compared to approaches that support integrated query optimization.

In addition, execution is partially sequential and may involve LLM-based components, including retrieval-augmented generation (RAG), which can introduce latency depending on the characteristics of individual sources and processing steps. As a result, execution efficiency is not solely determined by the query model, but also by the underlying data sources and components.

While the architecture is extensible through source-specific wrappers, enabling the integration of additional data sources, performance characteristics may vary across sources and do not scale uniformly.

Finally, the system combines deterministic execution with localized probabilistic components. While core steps such as parsing and query execution are reproducible, LLM-based translation and retrieval components (e.g., RAG) introduce limited variability. Execution traces mitigate this by exposing all intermediate steps, but full determinism cannot be guaranteed.

Generality of the Evaluation The findings of this study are most directly applicable to information-seeking tasks that require retrieving and combining data from multiple heterogeneous sources. In such scenarios, structured query interfaces like AQL provide advantages in terms of correctness, robustness, and accessibility.

However, the generalizability of the results is limited by the characteristics of the evaluated task set. Tasks involving complex cross-source dependencies, large-scale data integration, or highly exploratory analysis may require capabilities that are not fully supported by the current system design.

In addition, the evaluation is based on a prototype implementation and a limited set of data sources. While the underlying design principles are transferable, performance

characteristics may vary depending on the system configuration and the nature of the integrated sources.

The results therefore indicate that Rubicon is particularly well-suited for structured multi-source retrieval tasks, while its applicability to more complex or large-scale scenarios warrants further investigation.

6. Conclusion

6.1. Summary of Contributions

This thesis presented **Rubicon**, a system for transparent and interactive multi-source information retrieval. The core contribution is an operator-based query model that enables users to express retrieval tasks as explicit, composable steps across heterogeneous data sources.

System Design Rubicon introduces a structured query interface based on a small set of composable operators, allowing users to formulate and execute retrieval processes in a transparent and controllable manner. Instead of unified cross-source execution, the system adopts a stepwise model that prioritizes inspectability and user control over full automation.

Implementation A functional prototype was developed to demonstrate the approach's feasibility. The system integrates heterogeneous data sources through a wrapper-based architecture and combines deterministic query execution with selective LLM-based components, including retrieval-augmented generation (RAG)-style mechanisms.

Evaluation The proposed approach was evaluated in a controlled user study comparing AQL with a manual workflow. The results indicate that Rubicon improves task performance in terms of correctness and completeness, while also increasing robustness.

Efficiency effects depend on user expertise: experienced users benefit from faster execution, whereas less experienced users require additional interaction effort. At the same time, the system improves usability and reduces perceived workload, particularly for non-expert users.

Most notably, the results demonstrate a substantial reduction in dependency on prior technical expertise. The performance gap between higher- and lower-expertise users is substantially reduced, indicating that structured query interaction improves accessibility for complex retrieval tasks.

Key Insights The findings highlight three central insights for multi-source retrieval.

First, structuring retrieval into explicit steps improves correctness and robustness by enabling systematic exploration and refinement.

Second, interaction complexity is not eliminated but systematically redistributed: effort shifts from information discovery and tool coordination toward explicit query formulation.

Third, structured query execution represents a complementary alternative to LLM-driven approaches. While agent-based systems increase automation, they reduce transparency and predictability, whereas operator-based models provide control and inspectability at the cost of additional user effort.

These results demonstrate that transparent, operator-based query interfaces are a viable and effective approach for interactive retrieval in heterogeneous environments.

6.2. Future Work

Extensions of the Query Language A natural extension of the system is the support for multi-source queries with explicit join semantics. Future work could extend the query language to define relationships between data sources and enable cross-source joins.

This would require mechanisms for schema alignment, intermediate representations, and coordinated execution. While such extensions would increase expressiveness, they would also introduce additional complexity in query processing and reduce transparency.

Additional Data Sources The wrapper-based architecture facilitates the integration of additional data sources without changes to the core system. Future work should investigate how performance scales with increasing numbers of sources and more complex query workloads, particularly in scenarios requiring multi-step retrieval across heterogeneous environments.

Integration with Automated Agents An important direction is the integration of AQL with automated agent-based systems. Rather than treating structured queries and agent-based reasoning as competing paradigms, hybrid approaches could combine their strengths.

For example, agents could assist in query construction, suggest refinements, or generate intermediate steps, while execution remains transparent and controlled. This could reduce the initial learning effort while preserving the benefits of structured query execution.

Evaluation in Real-World Environments The current evaluation is based on a controlled user study with a limited number of participants and tasks. Future work should examine the applicability of the system in real-world environments, such as administrative workflows or enterprise data analysis settings.

Long-term studies with domain experts could provide insights into usage patterns, learning effects, and performance over time.

Overall, this work shows that structured, transparent retrieval is a viable alternative to fully automated LLM-based systems, particularly in complex multi-source environments where correctness, interpretability, and control are essential.

A. Appendix

A.1. User Interface

Query Interface

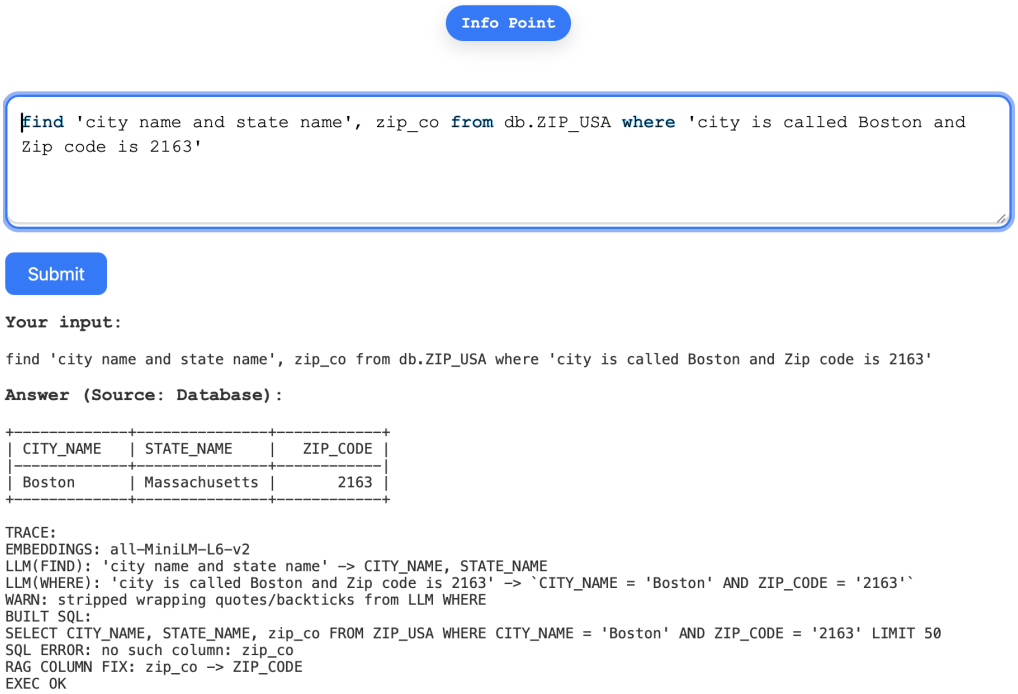


Figure A.1.: Web-based query interface displaying a user query, the resulting output, and the corresponding execution trace.

Information Panel

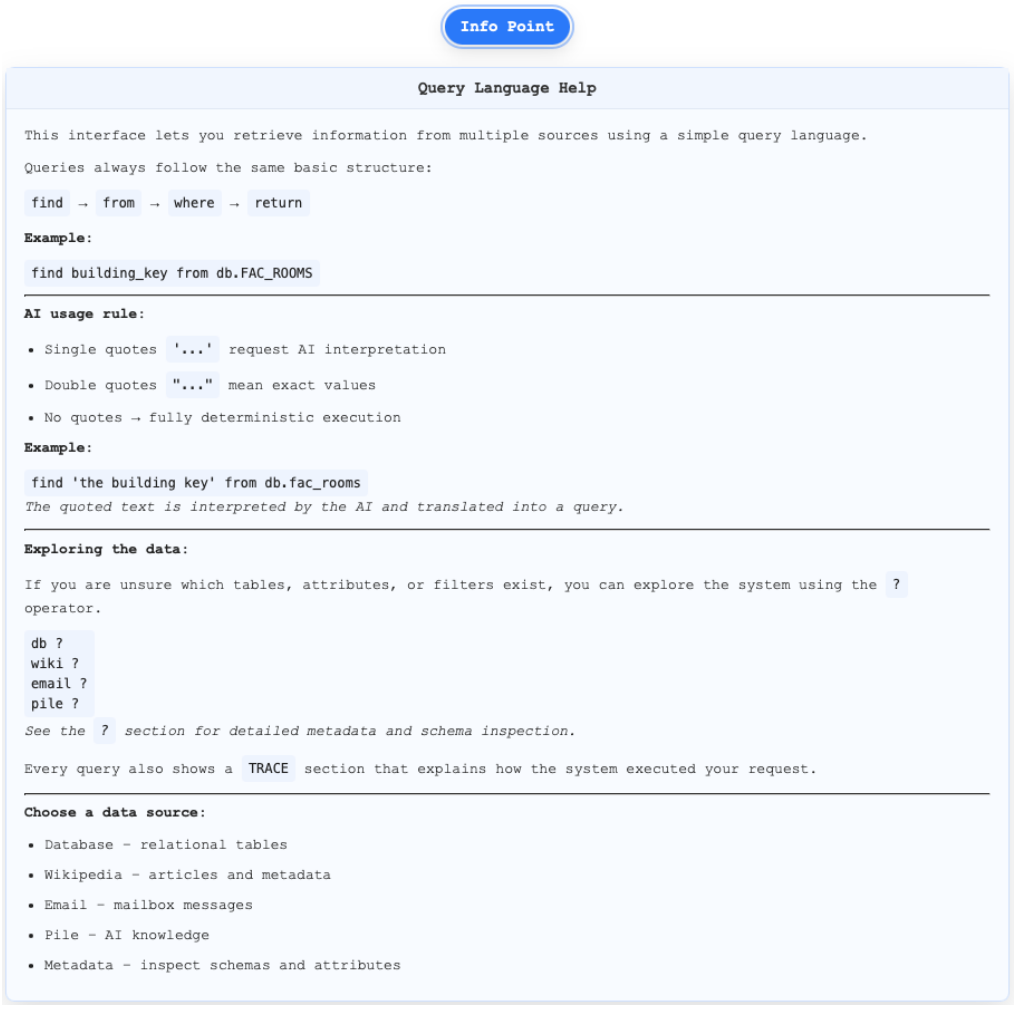


Figure A.2.: Integrated information panel providing structured information about data sources and query constructs.

A.2. Wrapper Execution Flows

Relational Database Wrapper

Figure A.3 shows the detailed execution flow of the relational database wrapper.

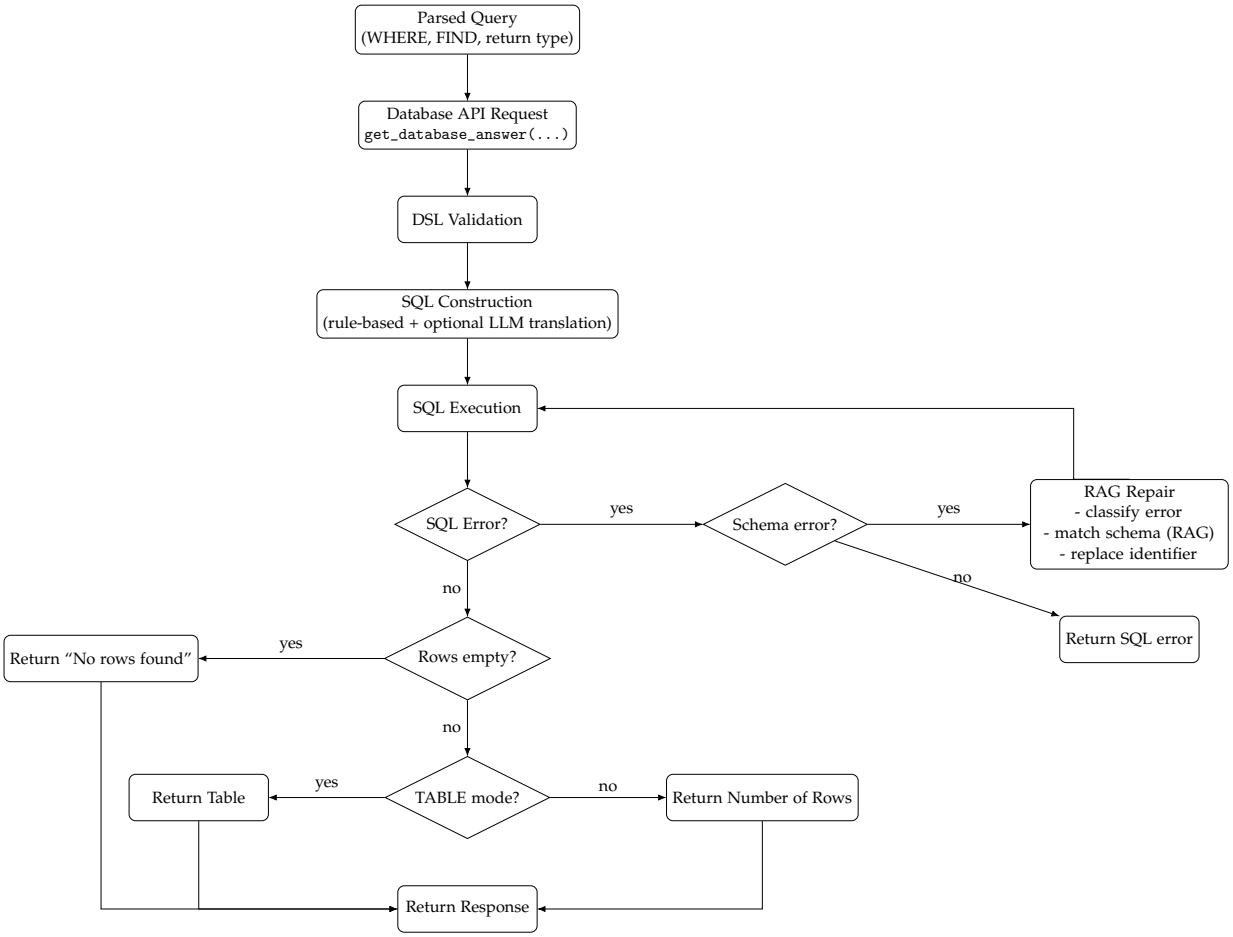


Figure A.3.: Execution flow of the database wrapper illustrating rule-based SQL construction, constrained LLM translation, and iterative schema repair.

Wikipedia Wrapper

Figure A.4 shows the detailed execution flow of the Wikipedia wrapper.

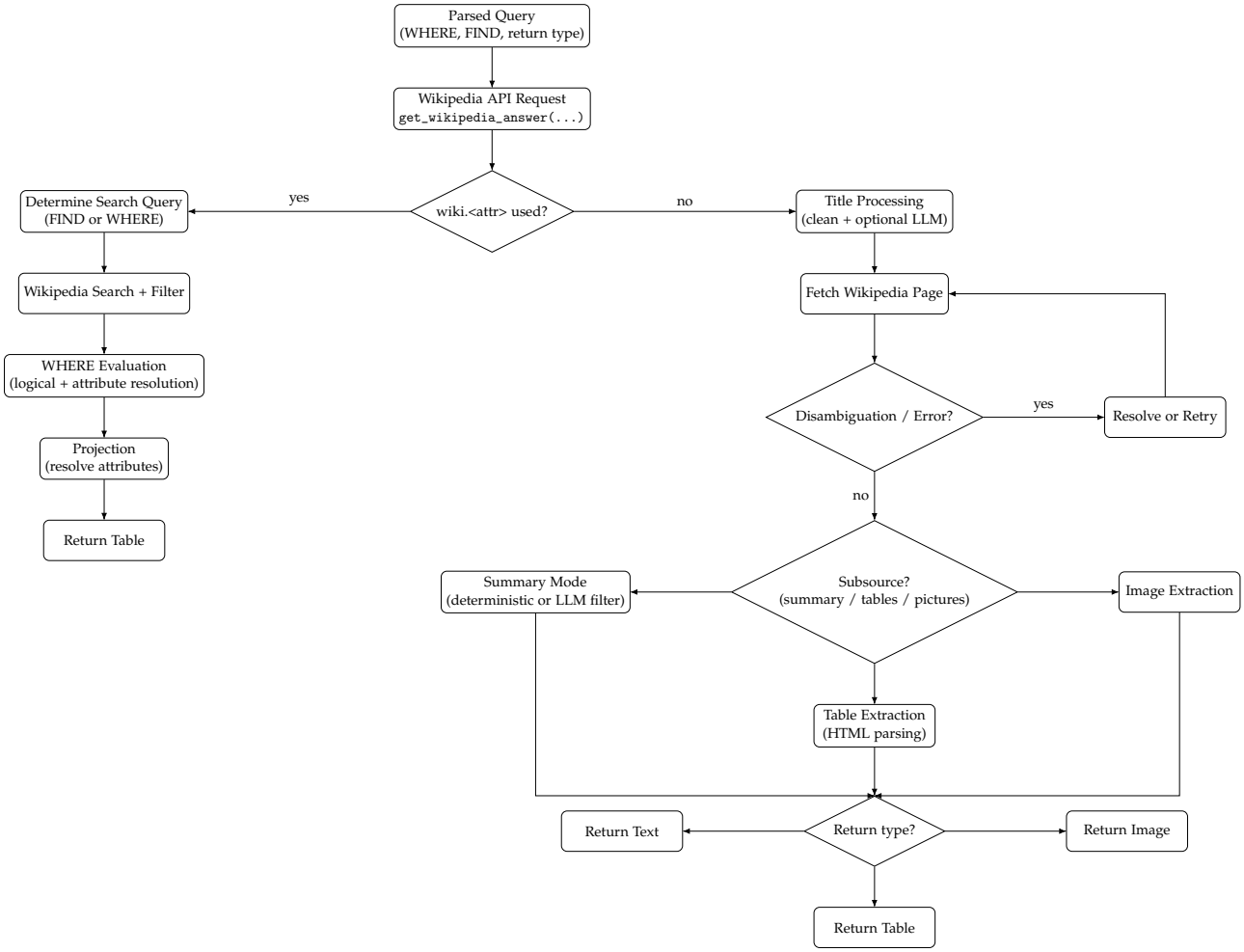


Figure A.4.: Execution flow of the Wikipedia wrapper illustrating attribute-based search and direct page retrieval modes.

Email Wrapper

Figure A.5 shows the detailed execution flow of the email wrapper.

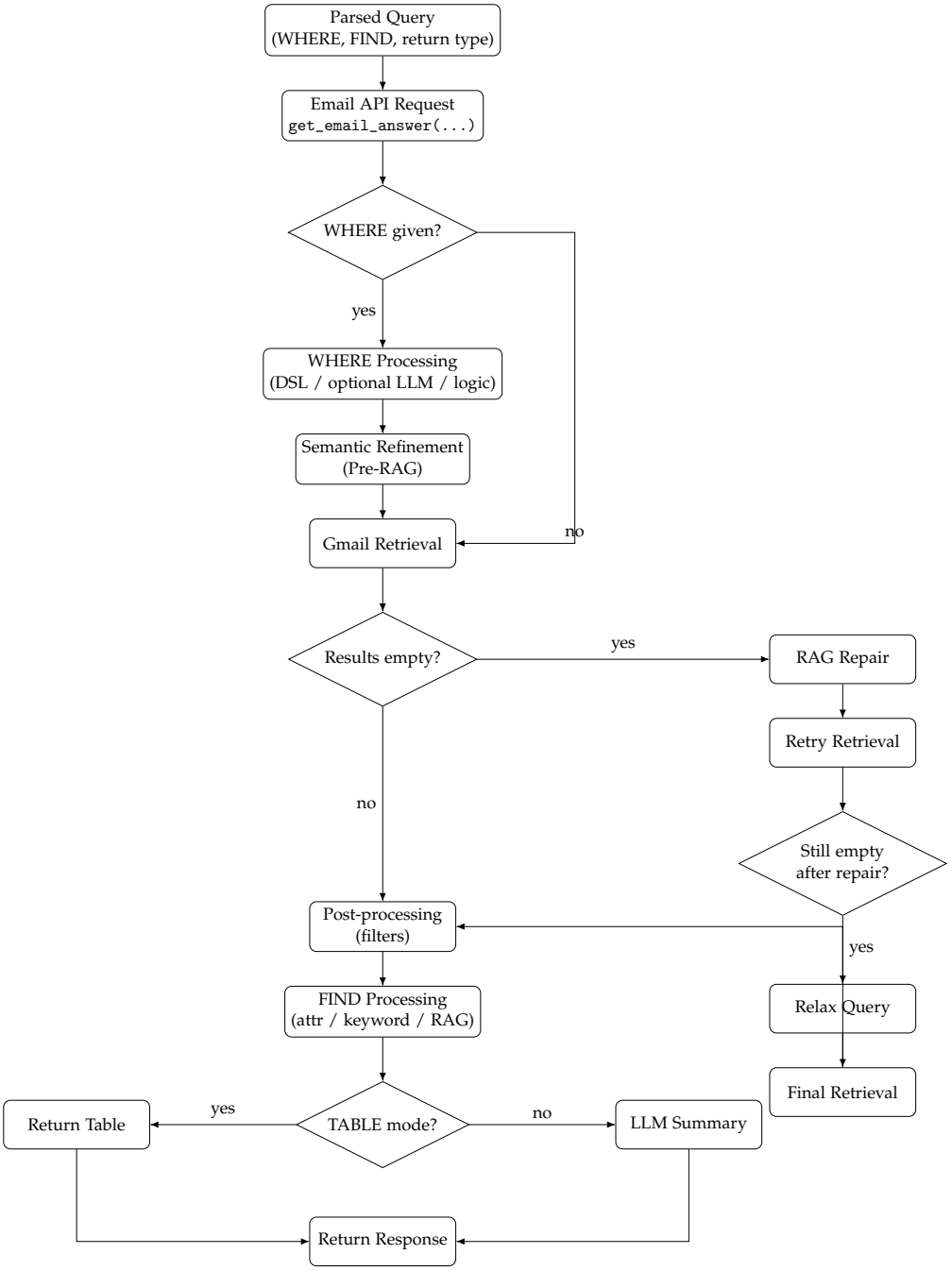


Figure A.5.: Execution flow of the email wrapper illustrating iterative refinement, repair, and post-processing stages.

LLM-Based Wrapper

Figure A.6 shows the detailed execution flow of the LLM-based wrapper.

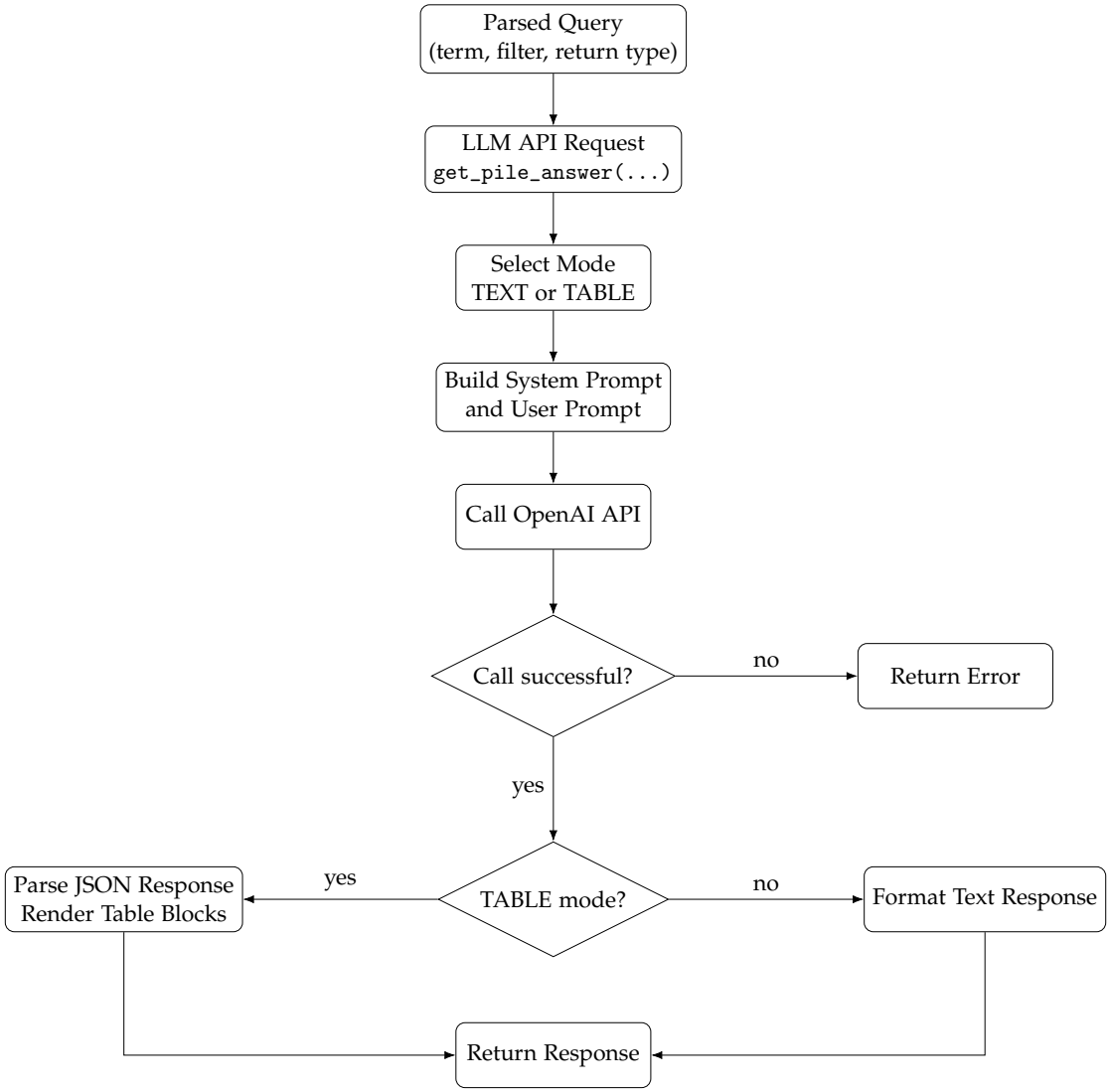


Figure A.6.: Execution flow of the LLM-based wrapper illustrating prompt construction and structured response handling.

A.3. Execution Trace Examples

Relational Database

```
[INPUT] find 'city name', zip_co from ZIP_USA
where not ('zip code is 623 or zip code is 624')
return table

[OUTPUT]
50 results returned (showing first 10 of 50 for brevity):

CITY_NAME | ZIP_CODE
---------------------
Alts el Maestro | 627
Camuy | 627
Sect Pena | 627
Urb del Carmen | 627
Urb Las Veredas | 627
Urb Linda Vista | 627
VISTAS de Camuy | 627
Adjuntas | 631
Castaner | 631
Rosario | 636
...

[TRACE]
EMBEDDINGS: all-MiniLM-L6-v2
LLM(FIND): 'city name' -> CITY_NAME
LLM(WHERE): 'zip code is 623 or zip code is 624' -> (ZIP_CODE = '623' OR
    ZIP_CODE = '624')
BUILT SQL:
SELECT CITY_NAME, zip_co FROM ZIP_USA
WHERE not ((ZIP_CODE = '623' OR ZIP_CODE = '624')) LIMIT 50
SQL ERROR: no such column: zip_co
RAG COLUMN FIX: zip_co -> ZIP_CODE
EXEC OK
```

Wikipedia

```
[INPUT] find wiki.title, wiki.url
from wikipedia
where "MIT" in wiki.abstract and "computer" in wiki.abstract
```

```
[OUTPUT]
title | url
------------------------------------------------------------------------
Massachusetts Institute of Technology | https://en.wikipedia.org/wiki/MIT

[TRACE]
WIKI: REQUEST RECEIVED
WIKI: SEARCH QUERY SOURCE = WHERE
WIKI: SEARCH MODE (NO LLM)
WIKI: SEARCH QUERY = 'MIT'
WIKI: NORMALIZED SEARCH QUERY = 'MIT'
WIKI: SEARCH RESULTS = 13
WIKI: LOAD PAGE 'Massachusetts Institute of Technology'
WIKI: LOGICAL AND
WIKI: APPLY WHERE ''MIT' in wiki.abstract'
WIKI: RESOLVE ATTRIBUTE 'abstract'
WIKI: WHERE RESULT -> MATCH
WIKI: APPLY WHERE ''computer' in wiki.abstract'
WIKI: RESOLVE ATTRIBUTE 'abstract'
WIKI: WHERE RESULT -> MATCH
WIKI: LOAD PAGE 'MIT License'
WIKI: LOGICAL AND
WIKI: APPLY WHERE ''MIT' in wiki.abstract'
WIKI: RESOLVE ATTRIBUTE 'abstract'
WIKI: WHERE RESULT -> MATCH
WIKI: APPLY WHERE ''computer' in wiki.abstract'
WIKI: RESOLVE ATTRIBUTE 'abstract'
WIKI: WHERE RESULT -> NO MATCH
WIKI: LOAD PAGE 'MIT Technology Review'
WIKI: LOGICAL AND
WIKI: APPLY WHERE ''MIT' in wiki.abstract'
WIKI: RESOLVE ATTRIBUTE 'abstract'
WIKI: WHERE RESULT -> MATCH
WIKI: APPLY WHERE ''computer' in wiki.abstract'
WIKI: RESOLVE ATTRIBUTE 'abstract'
WIKI: WHERE RESULT -> NO MATCH
WIKI: LOAD PAGE 'MIT (disambiguation)'
WIKI: PAGE LOAD FAILED 'MIT (disambiguation)' ("MIT (disambiguation)" may
    refer to:
Machakos Institute of Technology
Madras Institute of Technology
```

```
Maebashi Institute of Technology
Maharaja Institute of Technology
Maharashtra Institute of Technology
Mallabhum Institute of Technology
Mandalay Technological University
Manipal Institute of Technology
Manukau Institute of Technology
Mapua University
Marathwada Institute of Technology
Massachusetts Institute of Technology
Melbourne Institute of Technology
Milwaukee Area Technical College
Misamis Institute of Technology
Monterrey Institute of Technology and Higher Education
Muroran Institute of Technology
Musashi Institute of Technology
Muzaffarpur Institute of Technology
Iwate Menkoi Television
Made in Taiwan
Made in Thailand
Major Investigation Team
McIntire School of Commerce
Manzanillo International Terminal
Market if touched
MFS Investment Management
Master of Science in Information Technology
Mechanoid Invasion Trilogy
Melodic Intonation Therapy
Metal-insulator transition
Methylisothiazolinone
Ministero delle Infrastrutture e dei Trasporti
Ministry of Information Technology (disambiguation)
Mit (butterfly)
MIT BBS
MIT License
MIT: Murder Investigation Team
Monoiodotyrosine
Mujahidin Indonesia Timur
Multiple impact therapy
Myanmar Institute of Theology
National Intelligence Organization
Mitt (disambiguation))
```

```
WIKI: LOAD PAGE 'MIT Press'
WIKI: LOGICAL AND
WIKI: APPLY WHERE ''MIT' in wiki.abstract'
WIKI: RESOLVE ATTRIBUTE 'abstract'
WIKI: WHERE RESULT -> MATCH
WIKI: APPLY WHERE ''computer' in wiki.abstract'
WIKI: RESOLVE ATTRIBUTE 'abstract'
WIKI: WHERE RESULT -> NO MATCH
WIKI: LOAD PAGE 'MIT Open Learning'
WIKI: LOGICAL AND
WIKI: APPLY WHERE ''MIT' in wiki.abstract'
WIKI: RESOLVE ATTRIBUTE 'abstract'
WIKI: WHERE RESULT -> MATCH
WIKI: APPLY WHERE ''computer' in wiki.abstract'
WIKI: RESOLVE ATTRIBUTE 'abstract'
WIKI: WHERE RESULT -> NO MATCH
WIKI: LOAD PAGE 'A Mt'
WIKI: LOGICAL AND
WIKI: APPLY WHERE ''MIT' in wiki.abstract'
WIKI: RESOLVE ATTRIBUTE 'abstract'
WIKI: WHERE RESULT -> NO MATCH
WIKI: LOAD PAGE 'GmbH'
WIKI: LOGICAL AND
WIKI: APPLY WHERE ''MIT' in wiki.abstract'
WIKI: RESOLVE ATTRIBUTE 'abstract'
WIKI: WHERE RESULT -> MATCH
WIKI: APPLY WHERE ''computer' in wiki.abstract'
WIKI: RESOLVE ATTRIBUTE 'abstract'
WIKI: WHERE RESULT -> NO MATCH
WIKI: LOAD PAGE 'MIT Computation Center'
WIKI: LOGICAL AND
WIKI: APPLY WHERE ''MIT' in wiki.abstract'
WIKI: RESOLVE ATTRIBUTE 'abstract'
WIKI: WHERE RESULT -> MATCH
WIKI: APPLY WHERE ''computer' in wiki.abstract'
WIKI: RESOLVE ATTRIBUTE 'abstract'
WIKI: WHERE RESULT -> NO MATCH
WIKI: LOAD PAGE 'LemelsonMIT Prize'
WIKI: LOGICAL AND
WIKI: APPLY WHERE ''MIT' in wiki.abstract'
WIKI: RESOLVE ATTRIBUTE 'abstract'
WIKI: WHERE RESULT -> MATCH
```

```
WIKI: APPLY WHERE ''computer' in wiki.abstract'
WIKI: RESOLVE ATTRIBUTE 'abstract'
WIKI: WHERE RESULT -> NO MATCH
WIKI: LOAD PAGE 'MIT Media Lab'
WIKI: LOGICAL AND
WIKI: APPLY WHERE ''MIT' in wiki.abstract'
WIKI: RESOLVE ATTRIBUTE 'abstract'
WIKI: WHERE RESULT -> MATCH
WIKI: APPLY WHERE ''computer' in wiki.abstract'
WIKI: RESOLVE ATTRIBUTE 'abstract'
WIKI: WHERE RESULT -> NO MATCH
WIKI: LOAD PAGE 'MIT Blackjack Team'
WIKI: LOGICAL AND
WIKI: APPLY WHERE ''MIT' in wiki.abstract'
WIKI: RESOLVE ATTRIBUTE 'abstract'
WIKI: WHERE RESULT -> MATCH
WIKI: APPLY WHERE ''computer' in wiki.abstract'
WIKI: RESOLVE ATTRIBUTE 'abstract'
WIKI: WHERE RESULT -> NO MATCH
WIKI: LOAD PAGE 'University Park at MIT'
WIKI: LOGICAL AND
WIKI: APPLY WHERE ''MIT' in wiki.abstract'
WIKI: RESOLVE ATTRIBUTE 'abstract'
WIKI: WHERE RESULT -> MATCH
WIKI: APPLY WHERE ''computer' in wiki.abstract'
WIKI: RESOLVE ATTRIBUTE 'abstract'
WIKI: WHERE RESULT -> NO MATCH
WIKI: FILTERED RESULTS = 1
WIKI: RESOLVE ATTRIBUTE 'title'
WIKI: RESOLVE ATTRIBUTE 'url'
```

Email

```
[INPUT] find email.subject
from email
where (email.from = "google"
and not "sicherheitswarnung" in email.subject)
or ('email is from Kai')

[OUTPUT]
Subject:
-------------------------------------------------------------------------
```

```
- Fwd: TUM CIT Informatics: MINGA Mentoring Program - Register as Mentor!
- [Action Advised] Review Google Cloud credential security best practices
- Check your Google Account privacy settings
- [Product Update] Automatic enablement of new OpenTelemetry ingestion API
- rechnung fr hilfe der bachelorarbeit
- Zweite Test Mail
- Fwd: TUM Sustainability Day 29.04.26 | Einladung / Invitation
- Test mit Anhang
- Test

[TRACE]
EMAIL: REQUEST RECEIVED
GMAIL: AUTH START
GMAIL: TOKEN LOADED FROM FILE
GMAIL: TOKEN EXPIRED -> REFRESHING
GMAIL: TOKEN REFRESHED & SAVED
GMAIL: AUTH OK
CONTEXT: fetching recent inbox messages for semantic inference (Pre-RAG /
    Repair)
GMAIL: LIST IDS (limit=50, q=None)
GMAIL: LIST IDS returned 30 messages
EMAIL: OR handled via DNF split strategy
GMAIL: DNF branch search -> from:google -subject:sicherheitswarnung
GMAIL: LIST IDS (limit=50, q=from:google -subject:sicherheitswarnung)
GMAIL: LIST IDS returned 3 messages
LLM(WHERE): translating semantic clause ''email is from Kai''
LLM(WHERE): -> from:Kai
GMAIL: DNF branch search -> from:Kai
GMAIL: LIST IDS (limit=50, q=from:Kai)
GMAIL: LIST IDS returned 6 messages
GMAIL: GET MESSAGE 1/9
GMAIL: GET MESSAGE 2/9
GMAIL: GET MESSAGE 3/9
GMAIL: GET MESSAGE 4/9
GMAIL: GET MESSAGE 5/9
GMAIL: GET MESSAGE 6/9
GMAIL: GET MESSAGE 7/9
GMAIL: GET MESSAGE 8/9
GMAIL: GET MESSAGE 9/9
EMAIL: FETCHED 9 MESSAGES
FIND(ATTR): returning attributes ['subject']
```

LLM-Based

```
[INPUT] find climate change from pile where pollution

[OUTPUT]
Climate change is significantly influenced by pollution, particularly
   from greenhouse gas emissions such as carbon dioxide and methane.
These pollutants are primarily released through the burning of fossil
   fuels, industrial processes, and deforestation. The accumulation of
   these gases in the atmosphere leads to global warming, which affects
   weather patterns, sea levels, and ecosystems.
Addressing pollution is crucial for mitigating climate change and
   promoting environmental sustainability.

[TRACE]
PILE: REQUEST RECEIVED
PILE: MODE = TEXT
```

A.4. Participant Assignment Details

Figure A.7 shows the detailed participant assignment procedure.

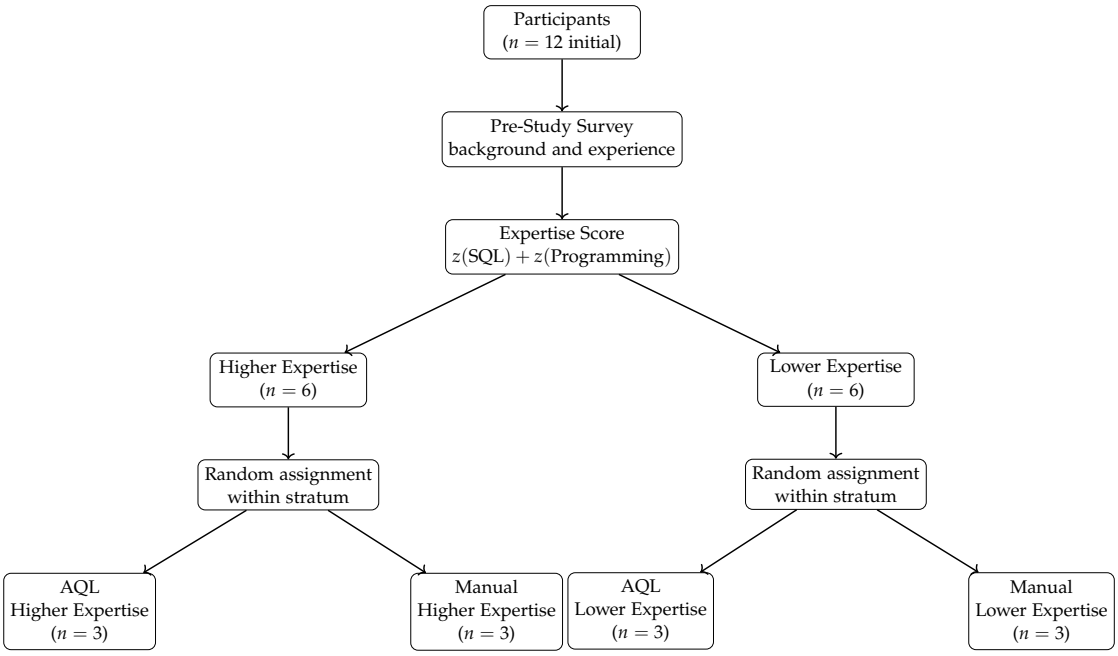


Figure A.7.: Overview of the participant assignment procedure. All participants first completed a pre-study survey. Based on self-reported SQL and programming proficiency, an expertise score was computed and used to stratify participants into higher- and lower-expertise groups. Within each stratum, participants were randomly assigned to the AQL or Manual condition, resulting in four balanced study groups. All participants completed the pre-study; two were excluded from the main analysis.

Table A.1 shows the final participant assignment.

Table A.1.: Final participant assignment after stratified randomization.

Participant	SQL	Programming	Expertise Score	Expertise Group	Condition
P8	5	5	2.71	Higher	AQL
P10	4	5	2.11	Higher	AQL
P7	3	4	0.96	Higher	AQL
P11	5	5	2.71	Higher	Manual
P12	4	4	1.55	Higher	Manual
P1	2	3	-0.20	Higher	Manual
P9	1	4	-0.24	Lower	AQL
P3	1	1	-1.92	Lower	AQL
P5	1	1	-1.92	Lower	AQL
P2	1	1	-1.92	Lower	Manual
P4	1	1	-1.92	Lower	Manual
P6	1	1	-1.92	Lower	Manual

A.5. Task Descriptions

Table A.2 lists all tasks used in both the user study and the system-level baseline evaluation.

Table A.2.: Overview of evaluation tasks (Q1–Q8).

Query	Description
Q1	List all distinct subjects offered by the university that have more than 100 enrolled students and are not offered as laboratory sections. Return the result as a table containing: the identifier of the subject, the title of the subject. Submit the resulting table as your final answer.
Q2	List all faculty buildings on campus that are classified as academic buildings, have an access level of 2, and have a gross area greater than 10,000. Return the result as a table containing: the name of the building, the type of the building, the gross area of the building. Submit the resulting table as your final answer.
Q3	Using the Pile dataset, retrieve a short general explanation of artificial intelligence in healthcare, including typical applications. Submit the retrieved text as your final answer.
Q4	Using the Pile dataset, identify articles related to implementation challenges of artificial intelligence. Submit the table as your final answer.
Q5	Which emails are labeled important, were received more than 30 days ago, and were not sent from Google? Return the result as a table containing: the subject of the email, the sender of the email, the date of the email. Submit the resulting table as your final answer.
Q6	Which emails satisfy one of the following conditions: the email was sent from Google after January 1, 2026, or the email was sent from Kai within the last 30 days? Additionally, the emails must not contain attachments, and the subject must not contain "privacy". Return the result as a table containing: the date of the email, the sender of the email, the recipient of the email, the subject of the email, whether the email contains attachments. Submit the resulting table as your final answer.
Q7	Which Wikipedia pages mention "rocket" in their abstract and belong to either the category "rockets" or the category "space launch vehicles"? Return the result as a table containing: the title of the page. Submit the resulting table as your final answer.
Q8	Which Wikipedia pages mention either "cancer" or "tumor" in their abstract, and include "who.int" among their sources? Return the result as a table containing: the title of the page, the page identifier, the URL of the page. Submit the resulting table as your final answer.

Note: Database tasks (Q1, Q2) are based on the Beaver dataset, a relational benchmark dataset commonly used for evaluating text-to-SQL systems.

A.6. Gold Answers

The gold-standard result sets were constructed using a two-step procedure.

First, candidate result sets were generated to obtain an initial pool of potentially relevant results under realistic system constraints.

Second, all candidate results were manually reviewed and validated to ensure correctness, completeness, and consistency.

The final gold-standard set, therefore, represents a manually verified reference independent of any single system output.

This ensures that all participants are evaluated against a consistent and reproducible reference set.

If additional valid results were identified during evaluation (e.g., results not included due to system constraints), these were manually verified and counted as correct predictions (true positives) for scoring, but were not added to the gold-standard set in order to maintain a consistent evaluation baseline.

This section summarizes the reference solutions used for evaluating task correctness. For structured queries, correctness is determined based on the returned tuples and attributes.

- **Q1 (DB-01):** Set of subjects where enrollment $>$ 100 and `IS_LAB_SECTION` $\neq$ `'Y'`.
- **Q2 (DB-02):** Set of academic buildings with `ACCESS_LEVEL_CODE = 2` and `EXT_GROSS_AREA` $>$ `10000`.
- **Q3 (LLM-01):** A concise explanatory text describing AI in healthcare, covering key aspects such as diagnostics, predictive analytics, and decision support, evaluated based on semantic correctness.
- **Q4 (LLM-02):** A fixed set of manually verified articles discussing implementation challenges of AI, represented as a table containing title, author, publication, and summary.
- **Q5 (Email-01):** Emails labeled important, older than 30 days, and not sent from Google (single matching email expected).
- **Q6 (Email-02):** Emails satisfying a disjunctive condition (Google after 01/01/2026 OR Kai within last 30 days), excluding attachments and subjects containing "privacy".
- **Q7 (WIKI-01):** Wikipedia pages containing "rocket" in the abstract and belonging to either the category "rockets" or "space launch vehicles".
- **Q8 (WIKI-02):** Wikipedia pages mentioning "cancer" or "tumor" and containing references to `who.int`.

A.7. User Study Results

Notably, individual performance varies within groups (e.g., P10), reflecting variability in user interaction and task strategies.

Table A.3.: Aggregated performance metrics per participant.

Participant	Group	Avg Time (s)	Avg F1	Avg Precision	Avg Recall	Dead End Rate
P1	Manual-High	776	0.48	0.53	0.45	0.38
P2	Manual-Low	207	0.29	0.63	0.24	0.38
P3	AQL-Low	542	0.81	0.95	0.75	0.00
P4	Manual-Low	428	0.51	0.75	0.43	0.25
P5	AQL-Low	589	0.88	1.00	0.84	0.00
P6	Manual-Low	491	0.42	0.54	0.37	0.38
P8	AQL-High	411	0.87	0.89	0.85	0.00
P9	AQL-Low	426	0.60	0.63	0.58	0.38
P10	AQL-High	189	0.23	0.25	0.21	0.75
P12	Manual-High	337	0.52	0.60	0.55	0.13

In addition to objective performance metrics, subjective measures related to perceived technical knowledge requirements, temporal demand, and expressiveness were collected. The following tables provide a detailed breakdown by participant group.

Table A.4.: Perceived need for advanced technical knowledge by group.

Group	Individual Responses	Average
AQL-High	3, 1	2.00
AQL-Low	2, 3, 4	3.00
Manual-High	1, 5	3.00
Manual-Low	5, 5, 5	5.00

Table A.5.: Perceived temporal demand (time pressure) by group.

Group	Individual Responses	Average
AQL-High	4, 1	2.50
AQL-Low	4, 3, 5	4.00
Manual-High	4, 5	4.50
Manual-Low	5, 4, 4	4.33

Table A.6.: Perceived expressiveness by group.

Group	Individual Responses	Average
AQL-High	2, 3	2.5
AQL-Low	4, 4, 4	4
Manual-High	2, 2	2
Manual-Low	2, 3, 3	2.67

A.8. Execution Details of the AQL Interface

Table A.7 shows execution times for queries using the AQL interface.

Table A.7.: Processing time of queries using the AQL interface with and without LLM support.

Query	Time (no LLM) [s]	Time (with LLM) [s]
Q1	1.80	5.30
Q2	0.01	2.70
Q3	–	2.01
Q4	–	6.40
Q5	4.10	4.30
Q6	7.50	6.60
Q7	15.30	–
Q8	21.20	–

Note: "–" indicates that the respective execution mode is not applicable. Tasks Q3 and Q4 rely exclusively on LLM-based retrieval, while Q7 and Q8 use deterministic execution. For comparison with LangChain, execution time is reported based on the actual execution mode used per query.

A.9. LangChain Execution Details

Table A.8 summarizes LangChain execution metrics.

LangChain exhibits substantial variability in processing time and token usage, particularly for database and Wikipedia queries requiring iterative tool interaction.

Note: Token counts include both prompt and completion tokens and reflect the full interaction history across tool calls.

Table A.8.: LangChain execution statistics per query.

Query	Time (s)	Tokens	Tool Calls	Result
Q1	141.33	600003	≥ 20	Wrong
Q2	29.66	27953	6	Correct
Q3	13.92	6677	1	Correct
Q4	8.61	3780	0	Wrong
Q5	9.53	6739	1	Correct
Q6	31.84	12486	2	Correct
Q7	47.83	32904	≥ 15	Correct
Q8	109.55	552064	≥ 20	Correct

A.10. Reproducibility and Artifacts

The implementation of the AQL system, evaluation scripts, and anonymized datasets used in this thesis are publicly available at:

`https://github.com/KaiA02/rubicon-aql-thesis`

The repository includes:

- the system implementation of Rubicon and the AQL interface,
- evaluation tasks and gold-standard definitions,
- study materials including pre-study, main-study, and post-study data.
- LangChain evaluation data.

All shared data has been anonymized to ensure compliance with privacy and research ethics guidelines.

List of Figures

3.1. System architecture and query processing flow, from user input to result generation. . . . 16
3.2. Simplified execution flow of the database wrapper, showing SQL construction, execution, optional repair, and result formatting. . . . 24
3.3. Simplified execution flow of the Wikipedia wrapper, illustrating attribute-based search and direct page retrieval modes. . . . 25
3.4. Simplified execution flow of the email wrapper, showing query interpretation, retrieval, refinement, post-processing, and result construction. . . . 26
3.5. Simplified execution flow of the LLM-based wrapper, showing prompt construction, model invocation, and response interpretation. . . . 27

4.1. Conceptual study design with interface condition as the between-subjects factor and participant expertise used for balanced group assignment and subsequent analysis. . . . 34
4.2. Accuracy versus execution time for AQL and LangChain. Points represent average values across all queries. . . . 45

A.1. Web-based query interface displaying a user query, the resulting output, and the corresponding execution trace. . . . 57
A.2. Integrated information panel providing structured information about data sources and query constructs. . . . 58
A.3. Execution flow of the database wrapper illustrating rule-based SQL construction, constrained LLM translation, and iterative schema repair. . . . 59
A.4. Execution flow of the Wikipedia wrapper illustrating attribute-based search and direct page retrieval modes. . . . 60
A.5. Execution flow of the email wrapper illustrating iterative refinement, repair, and post-processing stages. . . . 61
A.6. Execution flow of the LLM-based wrapper illustrating prompt construction and structured response handling. . . . 62
A.7. Overview of the participant assignment procedure. All participants first completed a pre-study survey. Based on self-reported SQL and programming proficiency, an expertise score was computed and used to stratify participants into higher- and lower-expertise groups. Within each stratum, participants were randomly assigned to the AQL or Manual condition, resulting in four balanced study groups. All participants completed the pre-study; two were excluded from the main analysis. . . . 70

List of Tables

4.1. Descriptive statistics of the variables used for expertise score construction. 37
4.2. Expertise distribution across experimental conditions. 38
4.3. Aggregated performance metrics by group. 41
4.4. Aggregated performance by interface condition. 41
4.5. Perceived usability, workload, and effectiveness by group (mean and SD). 42
4.6. Perceived usability, workload, and effectiveness by interface condition (mean and SD). 42

A.1. Final participant assignment after stratified randomization. 71
A.2. Overview of evaluation tasks (Q1–Q8). 72
A.3. Aggregated performance metrics per participant. 74
A.4. Perceived need for advanced technical knowledge by group. 74
A.5. Perceived temporal demand (time pressure) by group. 74
A.6. Perceived expressiveness by group. 75
A.7. Processing time of queries using the AQL interface with and without LLM support. 75
A.8. LangChain execution statistics per query. 76

Bibliography

[1] M. Stonebraker and U. Cetintemel. "One Size Fits All: An Idea Whose Time Has Come and Gone". In: *Proceedings of the 21st International Conference on Data Engineering (ICDE)*. 2005, pp. 2–11. DOI: 10.1109/ICDE.2005.1.

[2] A. Halevy, A. Rajaraman, and J. Ordille. "Data Integration: The Teenage Years". In: *Proceedings of the 32nd International Conference on Very Large Data Bases (VLDB)*. 2006, pp. 9–16.

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Proceedings of the 34th International Conference on Neural Information Processing Systems*. 2020. DOI: 10.48550/arXiv.2005.11401.

[4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. *ReAct: Synergizing Reasoning and Acting in Language Models*. 2023.

[5] J. M. Hellerstein, M. Stonebraker, and J. Hamilton. "Architecture of a Database System". In: *Foundations and Trends in Databases* 1.2 (2007), pp. 141–259. DOI: 10.1561/1900000002.

[6] S. Amershi, D. Weld, M. Vorvoreanu, A. Fourney, B. Nushi, P. Collisson, J. Suh, S. Iqbal, P. N. Bennett, K. Inkpen, J. Teevan, R. Kikin-Gil, and E. Horvitz. "Guidelines for Human-AI Interaction". In: *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. 2019, pp. 1–13. DOI: 10.1145/3290605.3300233.

[7] D. Hawking. "Challenges in Enterprise Search". In: *Proceedings of the 15th Australasian Database Conference - Volume 27*. Australian Computer Society, Inc., 2004, pp. 15–24.

[8] C. D. Manning, P. Raghavan, and H. Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[9] J. P. Callan, Z. Lu, and W. B. Croft. "Searching Distributed Collections with Inference Networks". In: *Proceedings of the 18th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. Association for Computing Machinery, 1995, pp. 21–28. DOI: 10.1145/215206.215328.

[10] A. Broder. "A Taxonomy of Web Search". In: *SIGIR Forum* 36.2 (2002), pp. 3–10. DOI: 10.1145/792550.792552.

[11] R. Baeza-Yates and B. Ribeiro-Neto. *Modern Information Retrieval*. Addison-Wesley, 1999.

[12] A. P. Sheth and J. A. Larson. "Federated Database Systems for Managing Distributed, Heterogeneous, and Autonomous Databases". In: *ACM Comput. Surv.* 22.3 (1990), pp. 183–236. DOI: 10.1145/96602.96604.

[13] G. Wiederhold. "Mediators in the Architecture of Future Information Systems". In: *Computer* 25.3 (1992), pp. 38–49. DOI: 10.1109/2.121508.

[14] E. Rahm and P. Bernstein. "A Survey of Approaches to Automatic Schema Matching." In: *VLDB J.* 10.4 (2001), pp. 334–350. DOI: 10.1007/s007780100057.

[15] M. Lenzerini. "Data Integration: A Theoretical Perspective". In: *Proceedings of the Twenty-First ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*. 2002, pp. 233–246. DOI: 10.1145/543613.543644.

[16] Y. Zhu, H. Yuan, S. Wang, J. Liu, W. Liu, C. Deng, H. Chen, Z. Liu, Z. Dou, and J.-R. Wen. *Large Language Models for Information Retrieval: A Survey*. 2025. DOI: 10.1145/3748304. arXiv: 2308.07107.

[17] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang. *Retrieval-Augmented Generation for Large Language Models: A Survey*. 2024. arXiv: 2312.10997. URL: https://arxiv.org/abs/2312.10997.

[18] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih. "Dense Passage Retrieval for Open-Domain Question Answering". In: *Conference: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2020, pp. 6769–6781. DOI: 10.18653/v1/2020.emnlp-main.550.

[19] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung. "Survey of Hallucination in Natural Language Generation". In: *ACM Comput. Surv.* 55.12 (2023), pp. 1–38. DOI: 10.1145/3571730.

[20] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen. "A Survey on Large Language Model Based Autonomous Agents". In: *Frontiers of Computer Science* 18.6 (2024). DOI: 10.1007/s11704-024-40231-1.

[21] T. Schick, J. Dwivedi-Yu, R. Dessí, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom. "Toolformer: Language Models Can Teach Themselves to Use Tools". In: *Proceedings of the 37th International Conference on Neural Information Processing Systems*. 2023.

[22] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. "Language Models Are Few-Shot Learners". In: *Proceedings of the 34th International Conference on Neural Information Processing Systems*. 2020.

[23] R. Nogueira and K. Cho. *Passage Re-ranking with BERT*. 2019.

[24] I. Ruthven. "Interactive Information Retrieval". In: *Annual Rev. Info. Sci & Technol.* 42.1 (2008), pp. 43–91.

[25] F. Doshi-Velez and B. Kim. *Towards A Rigorous Science of Interpretable Machine Learning*. 2017. arXiv: 1702.08608.

[26] LangChain. *LangChain Overview*. Accessed: 2026-03-27. 2026. URL: https://docs.langchain.com/oss/python/langchain/overview.

[27] OpenClaw. *OpenClaw Documentation*. Accessed: 2026-03-27. 2026. URL: https://docs.openclaw.ai/.

[28] deepset. *Introduction to Haystack*. Accessed: 2026-03-27. 2026. URL: https://docs.haystack.deepset.ai/docs/intro.

[29] LlamaIndex. *LlamaIndex GitHub Repository*. Accessed: 2026-03-27. 2026. URL: https://github.com/run-llama/llama_index.

[30] M. Fowler. *Domain-Specific Languages*. Addison-Wesley, 2011.

[31] C. Molnar, G. Casalicchio, and B. Bischl. "Interpretable Machine Learning – A Brief History, State-of-the-Art and Challenges". In: *ECML PKDD 2020 Workshops*. Springer International Publishing, 2020, pp. 417–431. DOI: 10.1007/978-3-030-65965-3_28.

[32] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell. "On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?" In: *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency (FAccT)*. 2021, pp. 610–623. DOI: 10.1145/3442188.3445922.

[33] N. Reimers and I. Gurevych. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. 2019.

[34] MIT 6.813 Human-Computer Interaction. *Reading 11: Experiment Design*. https://web.mit.edu/6.813/www/sp16/classes/11-experiment-design/. Accessed: 2026. 2016.

[35] J. Lazar, J. Feng, and H. Hochheiser. *Research Methods in Human-Computer Interaction*. Elsevier, 2017.

[36] S. Gathani, Z. Liu, P. J. Haas, and C. Demiralp. "What-if Analysis for Business Professionals: Current Practices and Future Opportunities". In: *Proceedings of the CHI Conference on Human Factors in Computing Systems*. 2025. DOI: 10.1145/3706598.3713672.

[37] S. Kandel, A. Paepcke, J. M. Hellerstein, and J. Heer. "Enterprise Data Analysis and Visualization: An Interview Study". In: *IEEE Transactions on Visualization and Computer Graphics* 18.12 (2012), pp. 2917–2926. DOI: 10.1109/TVCG.2012.219.

[38] J. Brooke. "SUS: A Quick and Dirty Usability Scale". In: *Usability Eval. Ind.* (1996), pp. 189–194.

[39] S. G. Hart and L. E. Staveland. “Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research”. In: *Human Mental Workload*. North-Holland, 1988, pp. 139–183. DOI: 10.1016/S0166-4115(08)62386-9.